

# Algorithm Specification and Supporting Documentation

## SHUTTLE: A Lattice Based Digital Signature Scheme

Long Chen<sup>1</sup>, Yifan Ming<sup>1</sup>, Zihan Liu<sup>1</sup>, Si Gao<sup>1</sup>, Jipeng Zhang<sup>2</sup>, Cong Zhang<sup>3</sup>, Pengfei Chen<sup>3</sup>, Taiyu Wang<sup>3</sup>, Minxuan Mei<sup>3</sup>, Yutao Sun<sup>4</sup>, Guofeng Tang<sup>5</sup>, and Jianhua Hou<sup>6</sup>

<sup>1</sup>Institute of Software, Chinese Academy of Sciences, China

<sup>2</sup>National University of Singapore, Singapore

<sup>3</sup>Zhejiang University, China

<sup>4</sup>Beijing Electronic Science and Technology Institute, China

<sup>5</sup>Singapore Management University, Singapore

<sup>6</sup>Information Engineering University, China

April 29, 2026

### Abstract

This document provides the complete specification, design rationale, security analysis, and performance evaluation for SHUTTLE, submitted to the New Generation Commercial Cryptography (NGCC) algorithm call by the Commercial Cryptography Standard Research Institute.

SHUTTLE is a highly compact, post-quantum digital signature scheme based on the conservative Module Learning with Errors (MLWE) assumption. Designed within the Fiat-Shamir with aborts paradigm, it leverages an advanced iterative rejection sampling technique to achieve one of the smallest signature sizes among integer-based implementations. To address the inherent challenges of practical deployment, SHUTTLE introduces three core innovations: (1) a novel structural compression mechanism tailored for MLWE that significantly reduces the signature size; (2) a rejection-free iterative sampling approach that ensures zero-knowledge simulatability while completely eliminating timing side-channel vulnerabilities; and (3) logarithmic optimizations for transcendental functions, which drastically reduce computational overhead. As a result, SHUTTLE offers a highly competitive balance of compactness, robust security, hardware-friendly integer arithmetic, and worst-case optimal performance for both high-end and resource-constrained platforms.

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>3</b> |
| 1.1      | Main Features and Advantages . . . . .             | 3        |
| <b>2</b> | <b>Algorithm Description</b>                       | <b>4</b> |
| 2.1      | Preliminaries and Notation . . . . .               | 4        |
| 2.1.1    | Basic Notation . . . . .                           | 4        |
| 2.1.2    | Rings, Vectors, and Splittings . . . . .           | 5        |
| 2.1.3    | Modular Arithmetic and Bit Decomposition . . . . . | 5        |
| 2.1.4    | Number Theoretic Transform (NTT) . . . . .         | 5        |
| 2.2      | Data Encoding and Hash Functions . . . . .         | 5        |
| 2.2.1    | Base Bit Packing and Unpacking . . . . .           | 5        |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 2.2.2    | Message Hashing and Challenge Generation . . . . .                   | 6         |
| 2.3      | Scheme Specification . . . . .                                       | 6         |
| 2.3.1    | Key Generation . . . . .                                             | 6         |
| 2.3.2    | Signing . . . . .                                                    | 7         |
| 2.3.3    | Verifying . . . . .                                                  | 8         |
| 2.4      | Auxiliary Functions . . . . .                                        | 9         |
| 2.4.1    | Conversion Between Data Types . . . . .                              | 9         |
| 2.4.2    | Encodings and Decodings . . . . .                                    | 9         |
| 2.4.3    | Pseudorandom Sampling . . . . .                                      | 10        |
| 2.4.4    | Compression and Hint Mechanism . . . . .                             | 11        |
| 2.4.5    | RejectSampling . . . . .                                             | 12        |
| 2.4.6    | Gaussian Sampler Specification . . . . .                             | 13        |
| 2.4.7    | Entropy Coding via Range Asymmetric Numeral Systems (rANS) . . . . . | 15        |
| 2.5      | Correctness . . . . .                                                | 17        |
| 2.6      | Parameter Sets . . . . .                                             | 18        |
| <b>3</b> | <b>Design Rationale</b>                                              | <b>18</b> |
| 3.1      | Underlying Hard Problems . . . . .                                   | 19        |
| 3.2      | Parameter Selection Strategy . . . . .                               | 19        |
| 3.3      | Signing Failure and Restart Probability Analysis . . . . .           | 20        |
| <b>4</b> | <b>Security Statement and Analysis</b>                               | <b>22</b> |
| 4.1      | Theoretical Security . . . . .                                       | 22        |
| 4.1.1    | CID . . . . .                                                        | 22        |
| 4.1.2    | error analysis . . . . .                                             | 22        |
| 4.1.3    | UF-CMA and sUF-CMA Security . . . . .                                | 24        |
| 4.1.4    | UF-NMA Security . . . . .                                            | 29        |
| 4.2      | Concrete Security Estimates . . . . .                                | 30        |
| 4.3      | Side-Channel Resistance and Implementation Security . . . . .        | 30        |
| 4.4      | Other Security Properties . . . . .                                  | 30        |
| <b>5</b> | <b>Performance Evaluation and Implementation Analysis</b>            | <b>31</b> |
| 5.1      | Evaluation Methodology and Platforms . . . . .                       | 31        |
| 5.2      | Software Performance on x86 Processors . . . . .                     | 31        |
| 5.2.1    | Benchmarking Results (Intel Core i7) . . . . .                       | 31        |
| 5.3      | Performance on Embedded Systems (ARM Cortex-M4) . . . . .            | 32        |
| 5.3.1    | Benchmarking Results (STM32F4) . . . . .                             | 32        |
| 5.4      | Optimization Techniques . . . . .                                    | 32        |
| 5.4.1    | Vectorization with AVX2 (x86) . . . . .                              | 32        |
| 5.4.2    | Optimizations for 32-bit Embedded Systems . . . . .                  | 32        |
| 5.5      | Transmission and Storage Overhead . . . . .                          | 33        |
| <b>6</b> | <b>Features Statement</b>                                            | <b>33</b> |
| 6.1      | Innovation . . . . .                                                 | 33        |
| 6.2      | Compatibility and Real-World Applicability . . . . .                 | 33        |
| 6.3      | Flexibility and Scalability . . . . .                                | 33        |
| <b>A</b> | <b>Intellectual Property Statement</b>                               | <b>34</b> |
| <b>B</b> | <b>Test Vectors</b>                                                  | <b>34</b> |

# 1 Introduction

This document provides the complete specification, security analysis, and performance evaluation for SHUTTLE, a lattice-based digital signature scheme submitted to the New Generation Commercial Cryptography (NGCC) algorithm call. SHUTTLE is designed to achieve an optimal balance between high computational efficiency, extremely compact transmission sizes, and robust post-quantum security.

At its core, SHUTTLE builds upon the Fiat-Shamir with Aborts paradigm but introduces a paradigm shift in how rejection sampling and secret key structures are handled. We adopt a “Compactness through Iteration” design philosophy: by fundamentally redesigning the sampling control flow and aggressively compressing the response vectors, we achieve signature sizes that rival floating-point-based schemes without sacrificing the simplicity of integer arithmetic. Instead of relying on aggressive or non-standard assumptions to reduce key sizes, we leverage a highly efficient iterative sampling mechanism combined with structural compression. This approach allows SHUTTLE to achieve highly competitive bandwidth profiles perfectly suited for real-world network protocols (such as IPsec and TLS 1.3) without sacrificing the rigorous security guarantees required by NGCC.

## 1.1 Main Features and Advantages

The distinct advantages of SHUTTLE stem from three primary theoretical and engineering innovations.

**Standardized Security with Structural Compression.** In recent compact lattice-based signatures, extreme bandwidth reduction is often achieved by relying on non-standard assumptions (e.g., NTWE) that lack long-term cryptanalysis. SHUTTLE fundamentally changes this perspective by strictly basing its security on the well-studied Module Learning with Errors (MLWE) and Module Short Integer Solution (MSIS) problems. To overcome the inherent size penalty of the MLWE secret key structure—specifically, the extra components that must be masked during rejection sampling—we introduce a tailored structural compression technique. This technique aggressively compresses the  $\ell_2$ -norm of the response vector  $\mathbf{z}$ . By doing so, SHUTTLE reduces the combined public key and signature size by approximately 40% compared to ML-DSA (Dilithium), bringing it within 30% of floating-point-based schemes like Falcon. This pure integer-based approach achieves a dual benefit: it minimizes bandwidth by dropping as many bits as possible, while entirely avoiding the implementation complexities and vulnerabilities associated with floating-point arithmetic.

**Rejection-Free Iterative Sampling and Side-Channel Resilience.** Historically, Fiat-Shamir with aborts schemes suffer from severe timing side-channel risks during the rejection phase, as the rejection decisions are highly dependent on the secret key. Furthermore, this abort-heavy process leads to a long-tail execution time distribution, complicating real-time system design. SHUTTLE successfully bridges this divide by employing a novel rejection-free iterative sampling mechanism. Instead of strictly requiring the output distribution to be identical to the target distribution, we relax this condition and rigorously bound the Rényi divergence between the input and output distributions. This guarantees a *zero rejection rate* during the internal iterations. This inherent zero-rejection property provides a substantial engineering gain: it completely eliminates data-dependent timing variations, ensuring robust side-channel resilience, and it drastically improves worst-case performance (short-tail latency), providing the strict reliability required for hardware deployments and real-time cryptographic protocols.

**Logarithmic Optimization for Integer-Only Arithmetic.** A major computational bottleneck in advanced iterative sampling frameworks is the repeated evaluation of Gaussian dis-

tributions and heavy transcendental functions (e.g., exponential functions) during the rejection arithmetic. SHUTTLE addresses this by optimizing the Gaussian sampler and replacing the heavy exponential operations with highly efficient logarithmic evaluations. By leveraging the monotonicity of logarithmic functions and splitting single complex comparisons into multiple simpler ones, we reduce the computational cost from approximately 100 exponential operations per iteration to just 1 logarithmic operation. Crucially, the secret key is completely decoupled from these transcendental calculations. This integer-only, heavily streamlined arithmetic allows our portable reference C implementation to outperform Dilithium, and our AVX2-optimized implementation to achieve roughly a  $3\times$  speedup, seamlessly combining extreme compactness with exceptional software and hardware performance.

## 2 Algorithm Description

### 2.1 Preliminaries and Notation

This section defines the mathematical spaces, rings, basic notations, and fundamental algebraic transformations used throughout the specification of SHUTTLE.

#### 2.1.1 Basic Notation

|                                                  |                                                                                                                                                      |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\mathbb{N}$                                     | the set of nonnegative integers                                                                                                                      |
| $\mathbb{Z}$                                     | the ring of integers                                                                                                                                 |
| $\ \mathbf{x}\ _2$                               | the Euclidean norm of vector $\mathbf{x}$                                                                                                            |
| $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle$     | the inner product of vectors $\mathbf{x}_1$ and $\mathbf{x}_2$                                                                                       |
| $\Pr[\mathcal{E}]$                               | the probability that event $\mathcal{E}$ occurs                                                                                                      |
| $s$                                              | secret key, a vector of $m + l + 1$ ring elements, also denoted by $[1, \mathbf{s}_1, \mathbf{s}_2]^T$                                               |
| $\mathbf{s}_1$                                   | the second component of the secret key, a vector of $l$ ring elements                                                                                |
| $\mathbf{s}_2$                                   | the third component of the secret key, a vector of $m$ ring elements                                                                                 |
| $\mathbf{A}_{\text{gen}}$                        | the public matrix of $m \times l$ ring elements generated from $\text{seed}_A$                                                                       |
| $\mathbf{a}_{\text{gen}}$                        | the public offset vector of $m$ ring elements generated from $\text{seed}_A$                                                                         |
| $b$                                              | the public vector of $m$ ring elements satisfying $b = \mathbf{a}_{\text{gen}} + \mathbf{A}_{\text{gen}} \circ \mathbf{s}_1 + \mathbf{s}_2 \pmod{q}$ |
| $\mathbf{j}$                                     | a public vector in $R_q^m$ such that $2(\mathbf{a}_{\text{gen}} - b) + q\mathbf{j}$ is well-defined over the integers                                |
| $\mathbf{A}$                                     | the derived public matrix $\mathbf{A} = [2(\mathbf{a}_{\text{gen}} - b) + q\mathbf{j} \mid 2\mathbf{A}_{\text{gen}} \mid 2\mathbf{I}_m]$             |
| $\text{seed}_A, \text{seed}_{sk}, \text{seed}_c$ | seeds to generate $\mathbf{A}$ , $s$ , and $c$ respectively                                                                                          |
| $K, \text{seed}_y$                               | master seed and ephemeral seed to generate the masking vector $\mathbf{y}$                                                                           |
| $\mathbf{y}$                                     | the ephemeral masking vector in $R^{l+m+1}$                                                                                                          |
| $\mathbf{z}$                                     | the response vector, typically split into $(\mathbf{z}_1, \mathbf{z}_2)$                                                                             |
| $\mathbf{h}$                                     | the boolean verification hint vector                                                                                                                 |
| $\circ$                                          | point-wise (Hadamard) multiplication of polynomials or vectors in the NTT domain                                                                     |
| $\mu$                                            | message digest used for challenge generation                                                                                                         |
| $\mathbf{w}$                                     | commitment vector computed from the ephemeral vector $\mathbf{y}$                                                                                    |
| $c$                                              | challenge polynomial with fixed Hamming weight $\tau$                                                                                                |
| $\sigma$                                         | the generated signature                                                                                                                              |

### 2.1.2 Rings, Vectors, and Splittings

We work over the polynomial ring  $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ , where  $n$  is a power of 2 and  $q$  is a prime. Vectors over  $R_q$  are denoted by bold lowercase letters (e.g.,  $\mathbf{v}$ ), and matrices by bold uppercase letters (e.g.,  $\mathbf{A}$ ).

In our algorithms, we frequently concatenate or split vectors of polynomials. Let  $\mathbf{v} \in R_q^m$ . When we write  $(\mathbf{v}_1, \mathbf{v}_2) \leftarrow \mathbf{v}$ , where  $\mathbf{v}_1$  has dimension  $m_1$  and  $\mathbf{v}_2$  has dimension  $m_2$  (with  $m_1 + m_2 = m$ ), it means  $\mathbf{v}_1$  consists of the first  $m_1$  polynomials of  $\mathbf{v}$ , and  $\mathbf{v}_2$  consists of the remaining  $m_2$  polynomials.

### 2.1.3 Modular Arithmetic and Bit Decomposition

To support our compression techniques, we define operations to extract the high-order and low-order bits of elements in  $\mathbb{Z}_q$ . Let  $\alpha_h$  be a chosen compression parameter that divides  $2(q-1)$ .

For any element  $x \in \mathbb{Z}_q$ , we define its central remainder  $x \bmod^\pm \alpha_h$  as the unique integer  $r_0 \in (-\alpha_h/2, \alpha_h/2]$  such that  $x \equiv r_0 \pmod{\alpha_h}$ . We define the base decomposition functions as follows:

- **Low-order part (LowBits):**  $x_0 = \text{LowBits}(x, \alpha_h) := x \bmod^\pm \alpha_h$ . In our algebraic structure, the least significant bit is denoted as  $\text{LSB}(x)$ .
- **High-order part (HighBits):**  $x_h = \text{HighBits}(x, \alpha_h) := (x - x_0)/\alpha_h \pmod{q}$ . In pseudo-code, this is denoted by the rounding operation  $\lfloor x/\alpha_h \rfloor$ .

For polynomials and vectors, these functions are applied coefficient-wise.

### 2.1.4 Number Theoretic Transform (NTT)

To achieve high-performance implementations, polynomial multiplication in  $R_q$  is accelerated using the Number Theoretic Transform (NTT). We require  $q \equiv 1 \pmod{2n}$ , ensuring the existence of a primitive  $2n$ -th root of unity  $\zeta \in \mathbb{Z}_q$ .

For a polynomial  $a \in R_q$ , the forward NTT maps it to an array  $\hat{a} \in \mathbb{Z}_q^n$  where  $\hat{a}_i = \text{NTT}(a)_i = a(\zeta^{2i+1}) \pmod{q}$ . The inverse transformation is  $a_i = \text{NTT}^{-1}(\hat{a})_i = n^{-1} \sum_{j=0}^{n-1} \hat{a}_j \zeta^{-(2j+1)i} \pmod{q}$ .

The symbol  $\circ$  denotes point-wise multiplication in the NTT domain:  $\hat{c} = \hat{a} \circ \hat{b} \implies \hat{c}_i = \hat{a}_i \cdot \hat{b}_i \pmod{q}$ . The standard polynomial multiplication is efficiently computed as  $c = \text{NTT}^{-1}(\text{NTT}(a) \circ \text{NTT}(b))$ . In our scheme, the public matrix  $\mathbf{A}_{\text{gen}}$  is generated and stored directly in the NTT domain (denoted as  $\hat{\mathbf{A}}_{\text{gen}}$ ) to minimize computational overhead.

## 2.2 Data Encoding and Hash Functions

To ensure interoperability and exact reproducibility of test vectors, we strictly define the base encoding primitives and the cryptographic hashing mechanisms used to process arbitrary-length messages. The specific serialization formats for the public key, secret key, and signature will be detailed alongside their respective algorithms in Section ??.

### 2.2.1 Base Bit Packing and Unpacking

The fundamental building blocks for serialization are the generic bit-packing and unpacking functions, assuming a little-endian byte order convention.

- **ByteEncode<sub>d</sub>(f):** Let  $f$  be an array of  $n$  integers, where each integer  $f_i \in [0, 2^d - 1]$ . This function converts each integer  $f_i$  into its  $d$ -bit binary representation (least significant bit first). These  $n \times d$  bits are concatenated and grouped into blocks of 8 bits to form a byte array. If  $n \cdot d$  is not a multiple of 8, the final byte is padded with zero bits.

- **ByteDecode<sub>d</sub>(B)**: The inverse function of **ByteEncode<sub>d</sub>**. It takes a byte array  $B$ , reads it as a sequence of bits, and regroups every  $d$  bits into an integer in the range  $[0, 2^d - 1]$ , outputting the array of  $n$  integers  $f$ .

For a polynomial in  $R_q$ , **ByteEncode<sub>d</sub>** is applied to its  $n$  coefficients. For vectors of polynomials, it is applied sequentially to each component and the results are concatenated.

### 2.2.2 Message Hashing and Challenge Generation

In the Fiat-Shamir with aborts paradigm, the signature must cryptographically bind the message  $M$ , the public key  $pk$ , and the prover's commitment  $\mathbf{w}$ . To ensure domain separation and mitigate multi-target attacks, we define a robust hashing mechanism.

Let  $H$  denote a generic collision-resistant cryptographic hash function or an Extendable-Output Function (XOF). **In strict accordance with the NGCC submission guidelines**, our algorithmic design and implementations treat  $H$  as a black-box primitive. For the current submission, we utilize the unified cryptographic hash API provided by the institute (via `API_PKC.zip`). This modular design ensures that the underlying hash function can be seamlessly replaced by the ultimately standardized NGCC hash algorithm in subsequent evaluation rounds without requiring any modifications to the core signature logic.

The process of binding the message into a fixed-length challenge seed  $seed_c$  is defined as follows:

1. **Public Key Hash ( $tr$ )**: To bind the signature to the specific public key and prevent key-substitution attacks, we first compute a collision-resistant hash of the encoded public key:

$$tr \leftarrow H(\text{pkEncode}(pk))$$

2. **Message Digest ( $\mu$ )**: The arbitrary-length message  $M \in \{0, 1\}^*$  is hashed together with  $tr$  to form a fixed-length message digest  $\mu$ . This step can be pre-computed if the message is large, supporting streaming APIs:

$$\mu \leftarrow H(tr \parallel M)$$

3. **Challenge Seed ( $seed_c$ )**: During the signing algorithm, the prover generates a commitment vector  $\mathbf{w}$ . The challenge seed  $seed_c$  is derived by hashing the message digest  $\mu$  and the encoded commitment  $\mathbf{w}$ :

$$seed_c \leftarrow H(\mu \parallel \text{ByteEncode}(\mathbf{w}))$$

The fixed-length seed  $seed_c$  is subsequently used to deterministically sample the challenge polynomial  $c$  via the **SampleC** function. This hierarchical hashing structure ensures that any modification to the message, the public key, or the commitment will result in a completely different challenge, thereby guaranteeing the Strong Existential Unforgeability under Chosen Message Attacks (SUF-CMA).

## 2.3 Scheme Specification

### 2.3.1 Key Generation

The key generation algorithm **KeyGen** takes a seed  $\xi$  as input and outputs a public key and a secret key, both encoded as byte strings. The seed  $\xi$  is expanded as needed to produce a public seed  $seed_A$ , a secret seed  $seed_{sk}$ , and an auxiliary secret seed  $K$ .

Using  $seed_A$ , the public parameters  $\mathbf{a}_{\text{gen}}$  and  $\mathbf{A}_{\text{gen}}$  are pseudorandomly generated. Here,  $\mathbf{a}_{\text{gen}}$  is a public vector and  $\mathbf{A}_{\text{gen}}$  is a public matrix; in implementation,  $\mathbf{A}_{\text{gen}}$  is typically stored

or recomputed in NTT representation for efficiency in subsequent arithmetic. Using  $seed_{sk}$ , the secret vectors  $\mathbf{s}_1$  and  $\mathbf{s}_2$  are pseudorandomly sampled from the bimodal distribution specified by our scheme.

The public vector is then computed as

$$b = \mathbf{a}_{\text{gen}} + \mathbf{A}_{\text{gen}} \circ \mathbf{s}_1 + \mathbf{s}_2 \pmod{q}.$$

Accordingly, the public key consists of an encoding of  $(seed_A, b)$ , while the secret key consists of an encoding of the secret material needed for signing, including the sampled secret vectors and the auxiliary seed  $K$ .

For verification purposes, the public quantities  $(\mathbf{a}_{\text{gen}}, \mathbf{A}_{\text{gen}}, b)$  define the derived matrix

$$\mathbf{A} = [2(\mathbf{a}_{\text{gen}} - b) + q\mathbf{j} \mid 2\mathbf{A}_{\text{gen}} \mid 2\mathbf{I}_m],$$

so that the secret vector  $[1, \mathbf{s}_1, \mathbf{s}_2]^T$  satisfies the homogeneous relation

$$\mathbf{A} \circ [1, \mathbf{s}_1, \mathbf{s}_2]^T = \mathbf{0} \pmod{2q}.$$

As is standard practice, generating the public parameters from a seed rather than storing them explicitly significantly reduces the public-key size. Moreover, separating the expanded seeds ensures strict domain separation between public-parameter generation, secret sampling, and signing-time randomness derivation.

---

**Algorithm 1** KeyGen( $\xi$ )

---

```

1: Input: Seed  $\xi \in \mathbb{B}^{\frac{\lambda}{8}}$ 
2: Output: Encoded public key  $pk$  and encoded secret key  $sk$ 
3:  $\kappa \leftarrow 0$ 
4:  $\kappa \leftarrow \kappa + 1$ 
5:  $(seed_A, seed_{sk}, K) \leftarrow \text{ExpandSeeds}(\xi \parallel \text{IntegerToBytes}(m, 1) \parallel \kappa, 4 \cdot \frac{\lambda}{8})$   $\triangleright$  expand the input
   seed:  $seed_A \in \{0, 1\}^{\frac{\lambda}{8}}, seed_{sk} \in \{0, 1\}^{2 \cdot \frac{\lambda}{8}}, K \in \{0, 1\}^{\frac{\lambda}{8}}$ 
6:  $(\mathbf{a}_{\text{gen}}, \hat{\mathbf{A}}_{\text{gen}}) \leftarrow \text{ExpandA}(seed_A)$   $\triangleright$  generate  $\mathbf{a}_{\text{gen}}$  and  $\mathbf{A}_{\text{gen}}$  from  $seed_A$ , with  $\mathbf{A}_{\text{gen}}$  stored
   in NTT representation as  $\hat{\mathbf{A}}_{\text{gen}}$ 
7:  $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \text{ExpandS}(seed_{sk})$   $\triangleright$  generate  $\mathbf{s}_1$  and  $\mathbf{s}_2$  in coefficient representation
8:  $b \leftarrow \mathbf{a}_{\text{gen}} + \text{NTT}^{-1}(\hat{\mathbf{A}}_{\text{gen}} \circ \text{NTT}(\mathbf{s}_1)) + \mathbf{s}_2 \pmod{q}$   $\triangleright$  compute  $b = \mathbf{a}_{\text{gen}} + \mathbf{A}_{\text{gen}} \circ \mathbf{s}_1 + \mathbf{s}_2$ 
    $\pmod{q}$ 
9: if  $\|(1, \mathbf{s}_1, \mathbf{s}_2)\|_2 \geq B_k$  or  $\|(1, \mathbf{s}_1, \mathbf{s}_2)\|_2 \leq B'_k$  then
10:   goto 4
11: end if
12:  $pk \leftarrow \text{pkEncode}(seed_A, b)$ 
13:  $sk \leftarrow \text{skEncode}(seed_A, b, K, \mathbf{s}_1, \mathbf{s}_2)$ 
14: return  $(pk, sk)$ 

```

---

### 2.3.2 Signing

The signing algorithm **Sign** takes as input an encoded secret key  $sk$ , a message  $M$ , and optional randomness  $rnd$ , and outputs a signature  $\sigma$ . The secret key is first decoded to recover the public seed  $seed_A$ , the public vector  $b$ , the auxiliary secret seed  $K$ , and the secret vectors  $\mathbf{s}_1$  and  $\mathbf{s}_2$ . Using  $seed_A$ , the public parameters are regenerated, and the derived public matrix  $\hat{\mathbf{A}}$  is reconstructed as needed.

To ensure strong unforgeability and mitigate multi-target attacks, the message is hashed together with the hash of the public key to obtain the message digest  $\mu$ . Then, together with the auxiliary secret seed, the optional randomness input, and a counter,  $\mu$  is used to derive a

seed from which the ephemeral masking vector  $\mathbf{y}$  is sampled. From  $\mathbf{y}$ , the signer computes a public commitment  $\mathbf{w}$ , extracts the information required for challenge generation, and derives a challenge seed  $seed_c$ . The challenge  $c$  is then sampled from  $seed_c$ .

Next, the response vector  $\mathbf{z}$  is computed from  $\mathbf{y}$ , the challenge  $c$ , and the secret key material through the internal rejection-free sampling procedure. If  $\mathbf{z}$  does not satisfy the prescribed signing bound  $B_s$ , the process is repeated with fresh pseudorandomness. Once the bound test is passed, the response is written as  $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2)$ , and auxiliary information  $\mathbf{h}$  is generated from  $\mathbf{z}_2$  and  $\mathbf{w}$  to support verification. The final signature consists of an encoding of  $(seed_c, \mathbf{z}_1, \mathbf{h})$ . If the encoding fails, the procedure continues with the next iteration. As in other Fiat-Shamir with aborts lattice signatures, the signing procedure is deterministic once the secret key, the message, and the optional randomness input are fixed.

---

**Algorithm 2**  $\text{Sign}(sk, M, rnd)$

---

```

1: Input: Encoded secret key  $sk$ , message  $M$ , optional randomness  $rnd$ 
2: Output: Signature  $\sigma$ 
3:  $(seed_A, b, K, \mathbf{s}_1, \mathbf{s}_2) \leftarrow \text{skDecode}(sk)$  ▷ decode the secret key
4:  $(\mathbf{a}_{\text{gen}}, \mathbf{A}_{\text{gen}}) \leftarrow \text{ExpandA}(seed_A)$  ▷ regenerate  $\mathbf{a}_{\text{gen}}$  and  $\mathbf{A}_{\text{gen}}$  from  $seed_A$ , with  $\mathbf{A}_{\text{gen}}$  in NTT representation as  $\hat{\mathbf{A}}_{\text{gen}}$ 
5:  $\hat{\mathbf{A}} \leftarrow [\text{NTT}(2(\mathbf{a}_{\text{gen}} - b) + q\mathbf{j}) \mid 2\hat{\mathbf{A}}_{\text{gen}} \mid 2\hat{\mathbf{I}}_m]$  ▷ reconstruct the derived public matrix in NTT representation
6:  $tr \leftarrow \text{H}(\text{pkEncode}(seed_A, b))$  ▷ hash the public key for domain separation
7:  $\mu \leftarrow \text{H}(tr \parallel M)$  ▷ compute the message digest bound to the public key
8:  $\kappa \leftarrow 0$ 
9: while true do
10:    $seed_y \leftarrow \text{ExpandSeeds}(K \parallel rnd \parallel \mu \parallel \text{IntegerToBytes}(\kappa, 2), \frac{\lambda}{8})$ 
11:    $\kappa \leftarrow \kappa + 1$ 
12:    $\mathbf{y} \leftarrow \text{SampleY}(seed_y)$  ▷ sample the ephemeral masking vector
13:    $\mathbf{w} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\text{CompressY}(\mathbf{y}))) \bmod 2q$  ▷ compute the public commitment
14:    $\mathbf{w}_0 \leftarrow \text{LSB}(\mathbf{w})$ 
15:    $\mathbf{w}_h \leftarrow \lfloor \mathbf{w} / \alpha_h \rfloor$ 
16:    $seed_c \leftarrow \text{H}(\mathbf{w}_h \parallel \mathbf{w}_0 \parallel \mu \parallel seed_A)$  ▷ derive the challenge seed
17:    $c \leftarrow \text{SampleC}(seed_c)$  ▷ sample the challenge with  $\tau$  entries set to +1 and the rest to 0
18:    $\mathbf{z} \leftarrow \text{CompressY}(\text{RejectSample}(c, \mathbf{y}, s))$  ▷ compute the response vector
19:   if  $\|\mathbf{z}\|_2 \geq B_s$  then
20:     continue
21:   end if
22:    $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$  ▷ Decompose  $\mathbf{z}$  into the first  $l + 1$  entries and the subsequent  $m$  entries.
23:    $\mathbf{h} \leftarrow \text{MakeHint}(\mathbf{z}_2, \mathbf{w})$  ▷ generate auxiliary information for verification
24:    $\sigma \leftarrow \text{sigEncode}(seed_c, \mathbf{z}_1, \mathbf{h})$ 
25:   if  $\sigma \neq \perp$  then
26:     return  $\sigma$ 
27:   end if
28: end while

```

---

### 2.3.3 Verifying

The verification algorithm  $\text{Verify}$  takes as input an encoded public key  $pk$ , a message  $M$ , and a signature  $\sigma$ , and outputs either **accept** or **reject**. The public key is first decoded to recover the public seed  $seed_A$  and the public vector  $b$ , while the signature is decoded to recover the challenge seed  $seed_c$ , the first component  $\mathbf{z}_1$  of the compressed response, and the auxiliary information  $\mathbf{h}$ .

Using  $seed_A$ , the public parameters are regenerated as needed. The verifier recomputes the

message digest  $\mu$ , derives the challenge  $c$  from  $seed_c$ , and reconstructs the low-bit part  $\mathbf{w}'_0$  of the commitment from  $\mathbf{z}_1$  and  $c$ . It then reconstructs the relevant derived public matrix and uses the auxiliary information  $\mathbf{h}$  to recover the high-bit commitment information  $\mathbf{w}_h$  together with the second response component  $\mathbf{z}_2$ .

Finally, the verifier recomputes the challenge seed from the recovered commitment information and accepts if and only if the recomputed challenge seed matches the one contained in the signature and the reconstructed response vector  $(\mathbf{z}_1, \mathbf{z}_2)$  satisfies the verification bound  $B_v$ . In general,  $B_v$  may differ from the signing bound  $B_s$  due to the loss incurred by response compression. Similar to advanced Fiat-Shamir with aborts signatures (e.g., Dilithium, HAETAE), the hint is used only to assist the recovery of the commitment information required for re-deriving the challenge.

---

**Algorithm 3** Verify( $pk, M, \sigma$ )

---

```

1: Input: Encoded public key  $pk$ , message  $M$ , signature  $\sigma$ 
2: Output: accept or reject
3:  $(seed_A, b) \leftarrow \text{pkDecode}(pk)$  ▷ decode the public key
4:  $tr \leftarrow H(pk)$  ▷ hash the public key
5:  $\mu \leftarrow H(tr \parallel M)$  ▷ compute the message digest
6:  $(seed_c, \mathbf{z}_1, \mathbf{h}) \leftarrow \text{sigDecode}(\sigma)$  ▷ decode the signature
7:  $c \leftarrow \text{SampleC}(seed_c)$  ▷ sample the challenge
8:  $(\mathbf{a}_{\text{gen}}, \hat{\mathbf{A}}_{\text{gen}}) \leftarrow \text{ExpandA}(seed_A)$ 
9:  $\mathbf{w}'_0 \leftarrow \text{RecoverLowBits}(\mathbf{z}_1, c)$  ▷ reconstruct the low-bit part of the commitment
10:  $\hat{\mathbf{A}}_1 \leftarrow \lfloor \text{NTT}(2(\mathbf{a}_{\text{gen}} - b) + q\mathbf{j}) \rfloor 2\hat{\mathbf{A}}_{\text{gen}}$ 
11:  $(\mathbf{w}_h, \mathbf{z}_2) \leftarrow \text{UseHint}(\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1)$  ▷ recover high-bit commitment and second response component
12:  $seed'_c \leftarrow H(\mathbf{w}_h \parallel \mathbf{w}'_0 \parallel \mu \parallel seed_A)$  ▷ recompute the challenge seed
13: if  $seed_c = seed'_c$  and  $\|(\mathbf{z}_1, \mathbf{z}_2)\|_2 \leq B_v$  then
14:   return accept
15: else
16:   return reject
17: end if

```

---

## 2.4 Auxiliary Functions

This section explicitly defines the auxiliary data conversion and serialization functions used in the main protocol algorithms. These definitions ensure strict interoperability and exact reproducibility of the Known Answer Tests (KATs).

### 2.4.1 Conversion Between Data Types

The primary data type conversion used in our scheme is the conversion of integers to fixed-length byte arrays, which is essential for domain separation in deterministic seed expansion.

- **IntegerToBytes**( $x, \ell$ ): Takes a non-negative integer  $x$  and a desired byte length  $\ell$  as input, and outputs an  $\ell$ -byte array representing  $x$  in little-endian byte order. If  $x \geq 2^{8\ell}$ , the function outputs  $\perp$ . In our algorithms, this is primarily used to encode the loop counter  $\kappa$  and the dimension parameter  $m$ .

### 2.4.2 Encodings and Decodings

Building upon the base  $\text{ByteEncode}_d$  and  $\text{ByteDecode}_d$  functions defined in Section 2.2, we specify the exact byte layouts for the public key, secret key, and signature. Let  $\parallel$  denote byte array concatenation.

**Public Key Encoding (pkEncode / pkDecode):** The public key consists of the  $\frac{\lambda}{8}$ -byte seed  $seed_A$  and the polynomial vector  $b$ . Assuming the coefficients of  $b$  are bounded by  $q$ , they are packed using  $d_q = \lceil \log_2(q) \rceil$  bits per coefficient.

$$\text{pkEncode}(seed_A, b) := seed_A \parallel \text{ByteEncode}_{d_q}(b)$$

The **pkDecode** function simply reads the first  $\frac{\lambda}{8}$  bytes as  $seed_A$  and unpacks the remaining bytes into  $b$ .

**Secret Key Encoding (skEncode / skDecode):** The secret key contains the public key components, the master seed  $K$ , and the secret vectors  $\mathbf{s}_1$  and  $\mathbf{s}_2$ . Since  $\mathbf{s}_1$  and  $\mathbf{s}_2$  are sampled from a narrow distribution bounded by  $\eta$ , their coefficients are shifted by  $+\eta$  to become non-negative and packed using  $d_\eta = \lceil \log_2(2\eta + 1) \rceil$  bits.

$$\text{skEncode}(seed_A, b, K, \mathbf{s}_1, \mathbf{s}_2) := seed_A \parallel \text{ByteEncode}_{d_q}(b) \parallel K \parallel \text{ByteEncode}_{d_\eta}(\mathbf{s}_1 + \eta) \parallel \text{ByteEncode}_{d_\eta}(\mathbf{s}_2 + \eta)$$

**Signature Encoding (sigEncode / sigDecode):** A signature comprises the challenge seed  $seed_c$ , the compressed response vector  $\mathbf{z}_1$ , and the verification hint  $\mathbf{h}$ . Let  $d_z$  and  $d_h$  be the bit-lengths required to pack the coefficients of  $\mathbf{z}_1$  and  $\mathbf{h}$ , respectively.

$$\text{sigEncode}(seed_c, \mathbf{z}_1, \mathbf{h}) := seed_c \parallel \text{ByteEncode}_{d_z}(\mathbf{z}_1) \parallel \text{ByteEncode}_{d_h}(\mathbf{h})$$

If any coefficient of  $\mathbf{z}_1$  or  $\mathbf{h}$  exceeds its allowed valid range during encoding, the function returns  $\perp$ , triggering a rejection in the signing algorithm. The **sigDecode** function reverses this concatenation to parse the signature components.

### 2.4.3 Pseudorandom Sampling

**Expansion of the secret key.** The subroutine **ExpandS** deterministically derives the secret vectors  $\mathbf{s}_1$  and  $\mathbf{s}_2$  from the seed  $seed_{sk}$ . Concretely, every coefficient in  $\mathbf{s}_1$  and  $\mathbf{s}_2$  is sampled independently according to the centered binomial distribution with parameter  $\eta$ , using pseudorandom bits expanded from  $seed_{sk}$ .

---

#### Algorithm 4 **ExpandS**( $seed_{sk}$ )

---

```

1: Input: Seed  $seed_{sk}$ 
2: Output:  $\mathbf{s}_1$  and  $\mathbf{s}_2$ 
3:  $buf \leftarrow \text{ExpandSeeds}(seed_{sk}, \text{enough bytes})$  ▷ expand the seed into pseudorandom randomness
4: for  $i = 0$  to  $l - 1$  do
5:   for  $j = 0$  to  $n - 1$  do
6:      $u_j^{(i)} \leftarrow \text{SampleCBD}(buf, \eta)$ 
7:   end for
8: end for
9: for  $i = 0$  to  $m - 1$  do
10:  for  $j = 0$  to  $n - 1$  do
11:     $v_j^{(i)} \leftarrow \text{SampleCBD}(buf, \eta)$ 
12:  end for
13: end for
14:  $\mathbf{s}_1 \leftarrow (u^{(0)}, \dots, u^{(l-1)})$ 
15:  $\mathbf{s}_2 \leftarrow (v^{(0)}, \dots, v^{(m-1)})$ 
16: return  $(\mathbf{s}_1, \mathbf{s}_2)$ 

```

---

**SampleY** The internal sampling algorithm **SampleY** takes as input a seed  $seed_y$  and outputs a vector  $\mathbf{y}$ . Each coefficient of  $\mathbf{y}$  is sampled independently from the discrete Gaussian distribution over  $\mathbb{Z}$  with standard deviation  $\mathbf{r}$ . In implementation, the seed  $seed_y$  is expanded pseudorandomly to provide the randomness required by the Gaussian sampler.

---

**Algorithm 5**  $\text{SampleY}(seed_y)$

---

```

1: Input: Seed  $seed_y$ 
2: Output:  $\mathbf{y} \in R^{l+m+1}$ 
3: for  $i = 0$  to  $l + m$  do
4:   for  $j = 0$  to  $n - 1$  do
5:      $y_{i,j} \leftarrow \text{SampleDGauss}(seed_y, i, j, \mathbf{r})$  ▷ sample independently from  $D_{\mathbb{Z}, \mathbf{r}}$ 
6:   end for
7: end for
8: return  $\mathbf{y} = (y_0, \dots, y_{l+m})$ 

```

---

#### 2.4.4 Compression and Hint Mechanism

This subsection describes the compression mechanisms used to significantly reduce the signature size. The compression is applied in two stages: compressing the ephemeral masking vector and truncating the commitment using a hint mechanism.

**Compression of  $\mathbf{y}$ .** The compression of the response is enabled by a preceding stretching step. In our setting, the secret-key distribution is asymmetric: some components are significantly shorter than others. During the iterative rejection sampling, we apply the auxiliary subroutine **StretchS** to selectively stretch these shorter components.

Since the selected components are initially short, enlarging them has only a minor effect on the overall secret-key norm. Thus, this selective stretching changes the rejection behavior only slightly, while modifying the scale of the corresponding coordinates to enable a more compact representation. After the sampling procedure terminates, the subroutine **CompressY** compresses the selected components of  $\mathbf{y}$ . In this way, we obtain a smaller signature size while incurring only a limited norm overhead.

---

**Algorithm 6**  $\text{StretchS}(s)$

---

```

1: Input:  $s = (1, \mathbf{s}_1, \mathbf{s}_2)$ 
2: Output:  $s' = (\alpha_1, \mathbf{s}_1, \mathbf{s}_2)$ 
3: return  $s' = (\alpha_1, \mathbf{s}_1, \mathbf{s}_2)$ 

```

---



---

**Algorithm 7**  $\text{CompressY}(\mathbf{y})$

---

```

1: Input:  $\mathbf{y} = (\mathbf{y}^{(0)}, \mathbf{y}^{(1)}, \dots, \mathbf{y}^{(l+m)})$ 
2: Output:  $\mathbf{z} = (\mathbf{z}^{(0)}, \mathbf{z}^{(1)}, \dots, \mathbf{z}^{(l+m)})$ 
3:  $\mathbf{z} \leftarrow (\lfloor \mathbf{y}^{(0)} / \alpha_1 \rfloor, \mathbf{y}^{(1)}, \dots, \mathbf{y}^{(l+m)})$ 
4: return  $\mathbf{z}$ 

```

---

**Generation of the hint.** To avoid transmitting the full commitment  $\mathbf{w}$ , we only transmit its least significant bits and a highly compressible boolean hint  $\mathbf{h}$ . The subroutine **MakeHint** computes this auxiliary hint to compensate for the information loss caused by omitting the second part  $\mathbf{z}_2$  of the response from the signature.

The signer first forms  $\tilde{\mathbf{w}} = \mathbf{w} - 2\mathbf{z}_2 \bmod 2q$ . It then separately computes the high-order parts of  $\mathbf{w}$  and  $\tilde{\mathbf{w}}$  with respect to the compression parameter  $\alpha_h$ . The hint is defined as

their difference modulo  $2q/\alpha_h$ . This allows the verifier to later recover the exact high-order commitment information.

---

**Algorithm 8** MakeHint( $\mathbf{z}_2, \mathbf{w}$ )

---

```

1: Input:  $\mathbf{z}_2, \mathbf{w}$ 
2: Output:  $\mathbf{h}$ 
3:  $\tilde{\mathbf{w}} \leftarrow \mathbf{w} - 2\mathbf{z}_2 \bmod 2q$ 
4:  $\mathbf{w}_h \leftarrow \lfloor \mathbf{w}/\alpha_h \rfloor$ 
5:  $\tilde{\mathbf{w}}_h \leftarrow \lfloor \tilde{\mathbf{w}}/\alpha_h \rfloor$ 
6:  $\mathbf{h} \leftarrow \mathbf{w}_h - \tilde{\mathbf{w}}_h \bmod 2(q-1)$ 
7: return  $\mathbf{h}$ 

```

---

**Recovery using the hint.** The subroutine UseHint uses the transmitted hint to recover the high-order commitment information needed during verification. The verifier first recomputes the modified commitment corresponding to the transmitted partial response  $\mathbf{z}_1$ , extracts its high-order part, and combines it with the hint to recover the high-order part of the original commitment  $\mathbf{w}_h$ .

Together with the reconstructed least significant bit  $\mathbf{w}'_0$ , this yields an approximate version of the original commitment, from which the omitted response part  $\mathbf{z}_2$  can be recovered up to the tolerated short error.

---

**Algorithm 9** UseHint( $\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1, c$ )

---

```

1: Input:  $\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1, c$ 
2: Output:  $\mathbf{w}_h, \mathbf{z}_2$ 
3:  $\tilde{\mathbf{w}} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}}_1 \circ \text{NTT}(\mathbf{z}_1)) - qc\mathbf{j}$  ▷ recompute the modified commitment
4:  $\tilde{\mathbf{w}}_h \leftarrow \lfloor \tilde{\mathbf{w}}/\alpha_h \rfloor$  ▷ extract its high-order part
5:  $\mathbf{w}_h \leftarrow \mathbf{h} + \tilde{\mathbf{w}}_h \bmod 2(q-1)$  ▷ recover the high-order part of the original commitment
6:  $\mathbf{w}^{\text{app}} \leftarrow \alpha_h \mathbf{w}_h + \mathbf{w}'_0$  ▷ form an approximate commitment value
7:  $\mathbf{z}_2 \leftarrow (\mathbf{w}^{\text{app}} - \tilde{\mathbf{w}})/2$  ▷ recover the omitted response part up to the tolerated short error
8: return  $(\mathbf{w}_h, \mathbf{z}_2)$ 

```

---

## 2.4.5 RejectSampling

---

**Algorithm 10** R( $y, v$ )

---

**Require:** Input vector  $y$ , shift vector  $v$

**Ensure:** One of  $y - v, y + v$

```

1:  $r \leftarrow \mathcal{U}([0, 1))$ 
2:  $r \leftarrow$ 
3:  $flag \leftarrow 1$ 
4: for  $i = 0$  to  $\tau_r$  do
5:    $flag \leftarrow flag - []$  ▷
6: end for
7: return  $y + flag \cdot v$ 

```

---

---

**Algorithm 11** `RejectSample( $y, c, s$ )`

---

**Require:** Input vector  $y$ , challenge  $c$ , secret vector  $s$

**Ensure:** Output  $z$

```
1:  $z_0 \leftarrow y$ 
2:  $k \leftarrow 0$ 
3: for each monomial  $x^j$  in  $c$  do
4:    $k \leftarrow k + 1$ 
5:    $z_k \leftarrow R(z_{k-1}, \text{StretchS}(s)x^j)$ 
6: end for
7: return  $z_k$ 
```

---

### 2.4.6 Gaussian Sampler Specification

The subroutine `SampleDGauss( $\cdot, \mathbf{r}$ )` used in `SampleY` produces independent samples from the centered discrete Gaussian distribution  $D_{\mathbb{Z}, \mathbf{r}}$  over the integers, where  $\mathbf{r} = 101$  for the 128-bit security parameter set and  $\mathbf{r} = 149$  for the 256-bit security parameter set. We adopt the BLISS-style convolution sampler of [DDL13], in the variant formalized as Algorithm 12 of [Ros20]: a wide Gaussian over  $\mathbb{Z}$  is realized as  $z = k \cdot x + y$ , where  $x$  is drawn from a small one-sided Gaussian via a cumulative distribution table and  $y$  is drawn uniformly from  $\{0, \dots, k-1\}$ , followed by an exponential rejection step. Throughout this section we fix the convolution multiplier

$$k = 64, \quad \sigma_2 = \mathbf{r}/k,$$

so that  $\sigma_2 = 1.5781$  in mode SHUTTLE-128 and  $\sigma_2 = 2.3281$  in mode SHUTTLE-256. Both modes share the same algorithmic structure; only the cumulative distribution table associated with  $\sigma_2$  depends on the mode.

**Choice of the small standard deviation.** For a fixed wide standard deviation  $\mathbf{r}$ , the parameters  $(\sigma_2, k)$  trade off two competing costs. A larger  $\sigma_2$  enlarges the cumulative distribution table that backs the small-Gaussian sampler, while a smaller  $\sigma_2$  raises the worst-case exponent in the rejection test and thereby reduces the average acceptance rate. Concretely, when the uniform  $y$  is approximated by  $u \cdot k$  with  $u \sim U(0, 1)$ , the acceptance probability simplifies to

$$A(x, u) = \exp\left(-\frac{u^2 + 2ux}{2\sigma_2^2}\right),$$

which is independent of the convolution multiplier  $k$  and depends only on  $\sigma_2$ . Combining the symmetric extraction-randomness cost of the cumulative distribution table with the rejection cost dictated by  $\sigma_2$ , a search across  $(\sigma_2, k)$  pairs (subject to  $k$  being a power of two for cheap shifts) selects  $k = 64$  as the cost-optimal point in both modes, yielding the small standard deviations stated above. The resulting acceptance rates are approximately 0.798 for  $\mathbf{r} = 101$  and 0.854 for  $\mathbf{r} = 149$ .

**Cumulative distribution tables.** For each mode we precompute a reverse cumulative distribution table RCDT of length  $L$ , where  $L$  is the truncation index. We truncate at 11 standard deviations of  $\sigma_2$ , giving  $L = 17$  for  $\sigma_2 = 1.5781$  and  $L = 26$  for  $\sigma_2 = 2.3281$ . Each entry is stored with 93 bits of precision and represents

$$\text{RCDT}[i] = \left\lceil 2^{93} \cdot \Pr_{X \sim D_{\mathbb{Z}^+, \sigma_2}}[X \geq i+1] \right\rceil, \quad i = 0, \dots, L-1.$$

The values are listed in Tables 1 and 2 as triples of 31-bit limbs, so that the 93-bit number is recovered as  $\text{RCDT}[i] = \text{lo} + \text{mid} \cdot 2^{31} + \text{hi} \cdot 2^{62}$ .

**Base sampler.** Algorithm 12 samples a non-negative integer  $x$  approximately distributed as  $D_{\mathbb{Z}^+, \sigma_2}$  by a constant-time scan over the precomputed RCDT. The construction is the natural transposition of the BaseSampler of [Ros20, HPRR20] (and Algorithm 12 of [HPRR20, Section 3.9]-style Falcon documentation) to 93-bit precision. The scan compares a single 93-bit uniform integer  $u$  to every table entry and counts the indices for which  $u < \text{RCDT}[i]$ ; the count is the returned sample. To guarantee constant-time execution, the comparison is implemented via three branchless 31-bit comparisons that propagate a carry across the limb boundary.

---

**Algorithm 12** BaseSampler( $\sigma_2$ )

---

```

1: Input: The mode-dependent table RCDT of length  $L$  associated with  $\sigma_2$ .
2: Output: An integer  $x \in \{0, \dots, L\}$  approximately distributed as  $D_{\mathbb{Z}^+, \sigma_2}$ .
3:  $u \leftarrow \text{UniformBits}(93)$ 
4:  $x \leftarrow 0$ 
5: for  $i = 0$  to  $L - 1$  do
6:    $x \leftarrow x + \llbracket u < \text{RCDT}[i] \rrbracket$   $\triangleright$  constant-time 93-bit comparison
7: end for
8: return  $x$ 

```

---

**Discrete Gaussian sampler.** The full sampler SampleDGauss is given in Algorithm 13. It follows the BLISS-style convolution [Ros20, Algorithm 12]: it draws a small one-sided Gaussian  $x$  via BaseSampler, a uniform low part  $y \in \{0, \dots, k - 1\}$ , sets the candidate  $z = k \cdot x + y$ , and accepts it with the conditional probability  $\exp(-y(y + 2kx)/(2\mathbf{r}^2))$ . The exponential rejection compensates exactly for the difference between the convolved candidate distribution and the target  $D_{\mathbb{Z}^+, \mathbf{r}}$ . A folded-distribution correction halves the acceptance probability on the singular event  $z = 0$  to restore the correct two-sided density. A final uniform sign is applied to lift the half-line distribution to  $D_{\mathbb{Z}, \mathbf{r}}$ .

---

**Algorithm 13** SampleDGauss( $\mathbf{r}$ )

---

```

1: Input: Standard deviation  $\mathbf{r}$ , with  $k = 64$  and  $\sigma_2 = \mathbf{r}/k$ .
2: Output: A sample  $z \sim D_{\mathbb{Z}, \mathbf{r}}$ .
3: loop
4:    $x \leftarrow \text{BaseSampler}(\sigma_2)$   $\triangleright x \sim D_{\mathbb{Z}^+, \sigma_2}$ 
5:    $y \leftarrow \text{UniformBits}(\log_2 k)$ 
6:    $z \leftarrow k \cdot x + y$ 
7:    $p \leftarrow \exp(-y(y + 2kx) / (2\mathbf{r}^2))$ 
8:   with probability  $1 - p$  restart the loop
9:   if  $z = 0$  then
10:    with probability  $1/2$  restart the loop
11:   end if
12:    $b \leftarrow \text{UniformBits}(1)$ 
13:   return  $(-1)^b \cdot z$ 
14: end loop

```

---

The exponential in line 7 is evaluated with the integer-only routine ApproxExp that we instantiate with the GALACTICS toolkit; its precision parameters are chosen so that the rejection test introduces no statistically observable bias relative to the exact target.

**Distribution quality.** The output of Algorithm 13 differs from the ideal distribution  $D_{\mathbb{Z}, \mathbf{r}}$  only through (i) the truncation of RCDT, (ii) the 93-bit rounding of the table entries, and (iii) the finite-precision evaluation of the rejection probability. Combining these three sources, the

1025-th order Rényi divergence between the law of the sampler output and the ideal  $D_{\mathbb{Z}, \mathbf{r}}$  is bounded by

$$R_{1025}(\text{SampleDGauss}(\mathbf{r}) \parallel D_{\mathbb{Z}, \mathbf{r}}) \leq \begin{cases} 1 + 2^{-93.40}, & \mathbf{r} = 101, \\ 1 + 2^{-93.75}, & \mathbf{r} = 149. \end{cases}$$

Following the analysis framework of [HPRR20] this bound is comfortably below the threshold  $1 + 1/(4Q_{\text{bs}})$  required for a  $\lambda = 256$ -bit security claim under the largest signature query budget targeted by our parameter sets, so the sampler can be substituted for the ideal distribution in the security reduction without quantitative loss.

Table 1: Reverse cumulative distribution table for  $\sigma_2 = 1.5781$  (mode SHUTTLE-128,  $L = 17$ ). The 93-bit entry is  $\text{RCDT}[i] = \text{lo} + \text{mid} \cdot 2^{31} + \text{hi} \cdot 2^{62}$ .

| $i$ | lo (hex)   | mid (hex)  | hi (hex)   | $\Pr[X \geq i + 1]$    |
|-----|------------|------------|------------|------------------------|
| 0   | 0x451749F4 | 0x18ACAE70 | 0x4C57A8B4 | $5.96 \times 10^{-1}$  |
| 1   | 0x36957301 | 0x799737CE | 0x2214D42A | $2.66 \times 10^{-1}$  |
| 2   | 0x78E3EE86 | 0x594E7B92 | 0x0AF10BD9 | $8.55 \times 10^{-2}$  |
| 3   | 0x3F7F03BD | 0x4C1BE93F | 0x02763175 | $1.92 \times 10^{-2}$  |
| 4   | 0x42E4A820 | 0x6029EBF2 | 0x0061BFAE | $2.98 \times 10^{-3}$  |
| 5   | 0x12E96D88 | 0x2C3F1462 | 0x000A584A | $3.16 \times 10^{-4}$  |
| 6   | 0x25E3DAE8 | 0x5DA4C386 | 0x0000BDF6 | $2.26 \times 10^{-5}$  |
| 7   | 0x4FBF560B | 0x16C867C8 | 0x00000932 | $1.10 \times 10^{-6}$  |
| 8   | 0x6E585FDD | 0x590E5C56 | 0x0000004C | $3.57 \times 10^{-8}$  |
| 9   | 0x7A9810AE | 0x56D6F249 | 0x00000001 | $7.82 \times 10^{-10}$ |
| 10  | 0x68B3CA36 | 0x0327875A | 0x00000000 | $1.15 \times 10^{-11}$ |
| 11  | 0x3BA50C27 | 0x0007F2CA | 0x00000000 | $1.13 \times 10^{-13}$ |
| 12  | 0x27184C49 | 0x00000D6B | 0x00000000 | $7.45 \times 10^{-16}$ |
| 13  | 0x1643B825 | 0x0000000F | 0x00000000 | $3.29 \times 10^{-18}$ |
| 14  | 0x05BEA231 | 0x00000000 | 0x00000000 | $9.73 \times 10^{-21}$ |
| 15  | 0x0002E981 | 0x00000000 | 0x00000000 | $1.93 \times 10^{-23}$ |
| 16  | 0x000000FC | 0x00000000 | 0x00000000 | $2.54 \times 10^{-26}$ |

#### 2.4.7 Entropy Coding via Range Asymmetric Numeral Systems (rANS)

While the low-order bits of the response vector are distributed almost uniformly, the high-order bits  $\text{HighBits}(\mathbf{z}_1)$  and the verification hint  $\mathbf{h}$  follow highly concentrated, non-uniform distributions. To compress these components down to their Shannon entropy and minimize the final signature size, we employ the range Asymmetric Numeral Systems (rANS) coding technique. Unlike Huffman coding, rANS operates efficiently with finite-precision integer arithmetic and achieves near-optimal compression rates even when symbol probabilities are not powers of two.

**rANS Parameters and Functions:** Let  $t > 0$  be the precision parameter (typically  $t = 8$  or 16 for byte-aligned or half-word-aligned operations). Let  $S \subseteq [0, 2^t - 1]$  be the alphabet of symbols to be encoded. We define a frequency function  $g : [0, 2^t - 1] \rightarrow \mathbb{Z} \cap (0, 2^t]$  such that  $\sum_{x \in S} g(x) \leq 2^t$  and  $g(x) = 0$  for all  $x \notin S$ .

Based on the frequency  $g(s)$ , we define the Cumulative Distribution Function (CDF) and the symbol mapping function:

- $\text{CDF}(s) = \sum_{y=0}^{s-1} g(y)$  for  $s \in S$ .
- $\text{symbol}(y)$  outputs the unique symbol  $s \in S$  that satisfies  $\text{CDF}(s) \leq y < \text{CDF}(s + 1)$ .

Table 2: Reverse cumulative distribution table for  $\sigma_2 = 2.3281$  (mode SHUTTLE-256,  $L = 26$ ). The 93-bit entry is  $\text{RCDT}[i] = \text{lo} + \text{mid} \cdot 2^{31} + \text{hi} \cdot 2^{62}$ .

| $i$ | lo (hex)   | mid (hex)  | hi (hex)   | $\Pr[X \geq i + 1]$    |
|-----|------------|------------|------------|------------------------|
| 0   | 0x27D8432F | 0x5CC00AED | 0x5A8CA8FB | $7.07 \times 10^{-1}$  |
| 1   | 0x6B59D50C | 0x52D4B77E | 0x38662F3F | $4.41 \times 10^{-1}$  |
| 2   | 0x0589FE19 | 0x52DE1A50 | 0x1E814196 | $2.38 \times 10^{-1}$  |
| 3   | 0x19228C60 | 0x3DBC9E12 | 0x0E2DC05D | $1.11 \times 10^{-1}$  |
| 4   | 0x395F2C65 | 0x64BD9992 | 0x059E9128 | $4.39 \times 10^{-2}$  |
| 5   | 0x21DD30A0 | 0x5D4446D4 | 0x01E35796 | $1.48 \times 10^{-2}$  |
| 6   | 0x53D9CE7A | 0x2423C1B7 | 0x0089147F | $4.18 \times 10^{-3}$  |
| 7   | 0x5BA66039 | 0x729CDEA1 | 0x0020B5B4 | $9.98 \times 10^{-4}$  |
| 8   | 0x4EF21306 | 0x73B4FA72 | 0x00068CFF | $2.00 \times 10^{-4}$  |
| 9   | 0x26A028CE | 0x7A947178 | 0x00011957 | $3.35 \times 10^{-5}$  |
| 10  | 0x5D3622B3 | 0x1C22D94B | 0x0000277B | $4.71 \times 10^{-6}$  |
| 11  | 0x6FED0352 | 0x06B4278A | 0x000004A1 | $5.52 \times 10^{-7}$  |
| 12  | 0x75A0969D | 0x7E04A1C3 | 0x00000073 | $5.40 \times 10^{-8}$  |
| 13  | 0x3CCE8069 | 0x3C0903B5 | 0x00000009 | $4.41 \times 10^{-9}$  |
| 14  | 0x713308E2 | 0x527DF437 | 0x00000000 | $3.00 \times 10^{-10}$ |
| 15  | 0x232E8D11 | 0x04ADABA1 | 0x00000000 | $1.70 \times 10^{-11}$ |
| 16  | 0x4F10AD3C | 0x003893F8 | 0x00000000 | $8.04 \times 10^{-13}$ |
| 17  | 0x60487ED3 | 0x000239C1 | 0x00000000 | $3.16 \times 10^{-14}$ |
| 18  | 0x462800E2 | 0x000012A8 | 0x00000000 | $1.04 \times 10^{-15}$ |
| 19  | 0x18E3CA18 | 0x00000082 | 0x00000000 | $2.82 \times 10^{-17}$ |
| 20  | 0x7A0221CE | 0x00000002 | 0x00000000 | $6.40 \times 10^{-19}$ |
| 21  | 0x07226B9E | 0x00000000 | 0x00000000 | $1.21 \times 10^{-20}$ |
| 22  | 0x001CADF0 | 0x00000000 | 0x00000000 | $1.90 \times 10^{-22}$ |
| 23  | 0x00005FE7 | 0x00000000 | 0x00000000 | $2.48 \times 10^{-24}$ |
| 24  | 0x0000010A | 0x00000000 | 0x00000000 | $2.69 \times 10^{-26}$ |
| 25  | 0x00000002 | 0x00000000 | 0x00000000 | $2.02 \times 10^{-28}$ |

The core state transition function  $C : \mathbb{Z} \times S \rightarrow \mathbb{Z}$  for encoding a symbol  $s$  into the current state  $x$  is defined as:

$$C(x, s) = \left\lfloor \frac{x}{g(s)} \right\rfloor \cdot 2^t + (x \bmod^+ g(s)) + \text{CDF}(s)$$

where  $\bmod^+$  denotes the non-negative remainder.

**Encoding and Decoding Algorithms:** The rANS encoder processes a sequence of symbols and updates a single natural number state. The decoder extracts the symbols in reverse order. In our scheme, the `Encode` and `Decode` subroutines used to compress the signature are defined as follows:

---

**Algorithm 14** Encode( $s_1, \dots, s_m \in S$ )

---

```
1: Input: A sequence of symbols  $(s_1, \dots, s_m)$ 
2: Output: Encoded state  $x_m \in \mathbb{Z}$ 
3:  $x_0 \leftarrow 0$ 
4: for  $i = 0$  to  $m - 1$  do
5:    $x_{i+1} \leftarrow C(x_i, s_{i+1})$ 
6: end for
7: return  $x_m$ 
```

---

---

**Algorithm 15** Decode( $x \in \mathbb{Z}, m \in \mathbb{N}$ )

---

```
1: Input: Encoded state  $x$ , number of expected symbols  $m$ 
2: Output: A sequence of symbols  $(s_1, \dots, s_m)$ 
3:  $y_m \leftarrow x$ 
4: for  $i = m$  down to 1 do
5:    $s_i \leftarrow \text{symbol}(y_i \bmod +2^t)$ 
6:    $y_{i-1} \leftarrow \lfloor y_i / 2^t \rfloor \cdot g(s_i) + (y_i \bmod +2^t) - \text{CDF}(s_i)$ 
7: end for
8: return  $(s_1, \dots, s_m)$ 
```

---

**Usage in the Signature Scheme:** During signature generation, after the high bits of the response and the hint are computed, their coefficients are mapped to the non-negative alphabet  $S$  and compressed using the **Encode** algorithm. The final signature incorporates the byte-encoded representation of the final state  $x_m$ . During verification, the verifier parses this state and uses **Decode** to exactly recover the high bits and the hint, which are then used to reconstruct the commitment.

## 2.5 Correctness

Before stating correctness, we note that correctness of our scheme is not immediate from the algebraic consistency of signing and verification alone. Since verification reconstructs the omitted response part only approximately from the transmitted hint and the least significant bit, acceptance additionally depends on the relation between the signing bound  $B_s$ , the verification bound  $B_v$ , and the compression parameter  $\alpha_h$ . Our proof follows the same line of argument as the compressed verification analysis in [Gär25, Lemma 5], translated into our notation.

We first state the two properties needed for the proof.

**Lemma 1** (Low-order reconstruction bound). *Assume that  $\alpha_h$  divides  $2(q-1)$ . Then for every commitment vector  $\mathbf{w}$ ,*

$$\|\text{LowBits}(\mathbf{w} \bmod 2q)\|_\infty \leq \alpha_h/2 + 2.$$

**Lemma 2** (Consistency of challenge reconstruction). *Let  $\sigma = (\text{seed}_c, \mathbf{z}_1, \mathbf{h})$  be a signature output by **Sign**, and let  $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$  be the corresponding decomposition of the response. Then the values reconstructed by **Verify** satisfy*

$$\mathbf{w}_h = \text{the high-order part of the commitment used in signing}$$

and

$$\mathbf{w}'_0 = \text{the least significant bit of the same commitment.}$$

Consequently, the recomputed challenge seed  $\text{seed}'_c$  equals the original challenge seed  $\text{seed}_c$ .

**Theorem 1** (Correctness). *Assume that  $B_v \geq B_s + \sqrt{nm}(\alpha_h/4 + 2)$ . Then, for every honestly generated key pair  $(pk, sk)$ , every message  $M$ , and every signature  $\sigma \leftarrow \text{Sign}(sk, M, rnd)$ , we have  $\text{Verify}(pk, M, \sigma) = \text{accept}$ .*

*Proof.* Let  $\sigma = (seed_c, \mathbf{z}_1, \mathbf{h})$  be a signature output by **Sign** on message  $M$ , and let  $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$  be the decomposition of the response vector computed during signing.

During signing, the signer computes the commitment  $\mathbf{w}$ , derives the challenge seed

$$seed_c = H(\mathbf{w}_h \| \mathbf{w}_0 \| \mu \| seed_A), \quad \mu = H(M),$$

and samples the challenge  $c = \text{SampleC}(seed_c)$ . The hint  $\mathbf{h}$  is generated from  $\mathbf{z}_2$  and  $\mathbf{w}$  so that the verifier can recover the same high-order commitment information. By the previous lemma, the verifier reconstructs the same high-order and low-order commitment information, and therefore recomputes the same challenge seed:  $seed'_c = seed_c$ . Hence the challenge check in verification always succeeds.

It remains to prove that the reconstructed response satisfies the verification bound. Let  $\mathbf{z}'_2$  denote the second response component reconstructed during verification. By the same argument as in [Gär25, Lemma 5], the verifier reconstructs  $\mathbf{z}'_2$  from the modified commitment, and the reconstruction error is determined by the low-order part of the commitment. More precisely,

$$\mathbf{z}'_2 = \mathbf{z}_2 + \frac{\Delta}{2} \pmod{\pm q},$$

where  $\Delta$  is the low-order reconstruction error. By the low-order bound above,

$$\|\Delta\|_{s_\infty} \leq \alpha_h/2 + 2.$$

Therefore,

$$\|\mathbf{z}'_2 - \mathbf{z}_2\|_2 \leq \sqrt{nm}(\alpha_h/4 + 2).$$

Using the triangle inequality, we obtain

$$\|(\mathbf{z}_1, \mathbf{z}'_2)\|_2 \leq \|(\mathbf{z}_1, \mathbf{z}_2)\|_2 + \sqrt{nm}(\alpha_h/4 + 2) \leq B_s + \sqrt{nm}(\alpha_h/4 + 2) \leq B_v.$$

Thus the reconstructed response always satisfies the verification bound, and the verification algorithm returns **accept**.  $\square$

## 2.6 Parameter Sets

Table 3 lists the proposed parameter sets targeting the required security categories.

Table 3: Proposed Parameter Sets and Security Levels

| Instance    | Classical Security | Quantum Security | Lattice Dim | n | Modulus | q   |
|-------------|--------------------|------------------|-------------|---|---------|-----|
| Algo-Level1 | 128-bit            | 80-bit           | ...         |   |         | ... |
| Algo-Level3 | 256-bit            | 128-bit          | ...         |   |         | ... |
| Algo-Level5 | 512-bit            | 256-bit          | ...         |   |         | ... |

## 3 Design Rationale

In accordance with the NGCC submission guidelines, this section outlines the fundamental design rationale of SHUTTLE, including the underlying mathematical hard problems, the strategy for parameter selection, and an analysis of the signing failure (restart) probability.

Table 4: Parametrizations with negligible rejection probability

| Security     | 128       | 256       | 512 | Description                                                |
|--------------|-----------|-----------|-----|------------------------------------------------------------|
| $n$          | 256       | 512       |     | Ring dimension                                             |
| $q$          | 13313     | 32257     |     | Modulus                                                    |
| $l$          | 3         | 3         |     | Number of polynomial components in $\mathbf{s}_1$          |
| $m$          | 2         | 2         |     | Number of polynomial components in $\mathbf{s}_2$          |
| $\eta_1$     | 1         | 1         |     | Centered binomial distribution parameter of $\mathbf{s}_1$ |
| $\eta_2$     | 1         | 1         |     | Centered binomial distribution parameter of $\mathbf{s}_2$ |
| $\mathbf{r}$ | 101       | 149       |     | parameter of $y$                                           |
| $\tau$       | 30        | 58        |     | Hamming weight of the challenge $c$                        |
| $\alpha_h$   | 128       | 1024      |     | Compression parameter for $h$                              |
| $\alpha_1$   | 8         | 16        |     | Compression parameter for the first component              |
| $B_k$        | 25.79     | 36.26     |     | Secret-key rejection bound                                 |
| $B_s$        | 3893.66   | 8391      |     | Signature rejection bound                                  |
| $B_v$        | 4663.0    | 16647     |     | Verification bound                                         |
| $M$          | $2^{-14}$ | $2^{-56}$ |     | Rejection rate                                             |
| pk size      | 928       | 1952      |     | Public key size                                            |
| Sign size    | 1160      | 2055      |     | Signature size                                             |
| Combined     | 2088      | 4007      |     | Total size                                                 |
| MLWE         | 131       | 276       |     | Estimated MLWE security                                    |
| MSIS         | 131       | 261       |     | Estimated MSIS security                                    |

### 3.1 Underlying Hard Problems

The security of SHUTTLE is fundamentally based on two well-studied lattice problems over the polynomial ring  $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ :

- **Module Learning with Errors (MLWE):** The pseudo-randomness of the public key relies on the hardness of the MLWE problem. We specifically choose MLWE over the non-standard NTWE problem used in [Gär25] to rely on more conservative and widely accepted security assumptions, offsetting the size penalty via our novel structural compression technique.
- **Module Short Integer Solution (MSIS):** The unforgeability of the signature relies on the hardness of the MSIS problem. An attacker attempting to forge a signature without knowing the secret key must find a short vector satisfying the verification equation, which reduces to solving the MSIS problem.

### 3.2 Parameter Selection Strategy

Our parameters are carefully chosen to balance extreme compactness, computational efficiency, and robust security. The parameter set must simultaneously satisfy the following three core requirements:

**Requirement 1: Hardness of SIS and LWE.** For a parameter set to be admissible, the hardness of the underlying SIS and LWE instances shall both provide at least  $\lambda$  bits of quantum security. These security levels are estimated using the standard lattice estimator. For the SIS instance, we evaluate the advantage  $Adv_{q,l,m,B_v}^{\text{SIS}}$  based on the verification bound  $B_v$ . For the

LWE instance, we evaluate the advantage  $Adv_{q,l,m,\chi_{s_1},\chi_{s_2}}^{\text{LWE}}$  based on the secret key distributions. The parameter set must satisfy:

$$\log_2\left(\frac{1}{Adv_{q,l,m,B_v}^{\text{SIS}}}\right) \geq \lambda \quad \text{and} \quad \log_2\left(\frac{1}{Adv_{q,l,m,\chi_{s_1},\chi_{s_2}}^{\text{LWE}}}\right) \geq \lambda.$$

The overall computational security level is determined by the smaller of the two estimates.

**Requirement 2: Challenge Space Size.** Let  $\mathcal{C}$  denote the challenge space. In this scheme, each challenge is represented by a polynomial with coefficients in  $\{0, 1\}$  and a fixed Hamming weight exactly  $\tau$ . Therefore, the number of possible challenges is  $|\mathcal{C}| = \binom{n}{\tau}$ . To prevent guessing attacks and ensure the zero-knowledge property in the Fiat-Shamir transform, the parameter set must satisfy:

$$\binom{n}{\tau} \geq 2^\lambda.$$

**Requirement 3: Relative Error of Iterative Rejection Sampling.** To ensure that our rejection-free iterative sampling does not leak side-channel information and remains statistically indistinguishable from the ideal distribution, the relative error  $\varepsilon_{\text{rel}}$  introduced by the iterations must be strictly bounded. The parameters shall satisfy:

$$\varepsilon_{\text{rel}} \leq 2^{-90}.$$

For the purpose of parameter selection,  $\varepsilon_{\text{rel}}$  is upper-bounded by:

$$\varepsilon_{\text{rel}} \leq \frac{2\alpha\sqrt{2\pi}\rho(\pi\alpha)}{\rho_\alpha(1)(1 - \rho_1(2\pi\alpha))}.$$

Accordingly, the parameter set is chosen so that this theoretical upper bound remains  $\leq 2^{-90}$ .

### 3.3 Signing Failure and Restart Probability Analysis

Unlike traditional Fiat-Shamir with aborts schemes (e.g., Dilithium) which experience high rejection rates during the masking phase, SHUTTLE employs a *rejection-free* iterative sampling technique. This means that the internal iterations for sign determination never abort, completely eliminating timing side-channels associated with secret-dependent rejections.

However, to ensure strict correctness and security bounds, the final signature must still pass a norm check (i.e.,  $\|(\mathbf{z}_1, \mathbf{z}'_2)\|_2 \leq B_s$ ). If the sampled commitment and the resulting response exceed this bound, the signing algorithm restarts. Because our parameters are tightly optimized to squeeze the signature size, this final norm check introduces a non-negligible restart probability (often referred to as the signing failure rate). We mathematically bound this probability to ensure that the expected number of repetitions per signature remains small (e.g.,  $\approx 2$  to 4 attempts). This deliberate trade-off allows us to achieve significantly smaller public keys and signatures while maintaining a short-tail execution time distribution, ensuring worst-case optimal performance in practical deployments.

### CID scheme

| $\mathbf{P}(\mathbf{A}, \mathbf{s})$                                                                                                                                                                                                                                                                                                                                                                           | $\mathbf{V}(\mathbf{A})$                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| $\mathbf{y} \leftarrow \mathbf{SampleY}$<br>$\mathbf{w} = \mathbf{A}\mathbf{y} \pmod{2q}$<br>$w_0 \leftarrow LSB(\mathbf{w})$                                                                                                                                                                                                                                                                                  |                                                                                      |
|                                                                                                                                                                                                                                                                                                                                                                                                                | $\xrightarrow{\mathbf{w}} c \leftarrow \mathcal{U}(\mathcal{C})$<br>$\xleftarrow{c}$ |
| $\mathbf{z}_0 = \mathbf{y}$<br><b>for</b> Each non-zero $c_i$ of $c$<br>$\mathbf{z}_i = R_{sc_i}(\mathbf{z}_{i-1})$<br>$k = \ \mathbf{c}\ _1$<br><b>if</b> $\mathbf{z}_k = \perp$ : <b>Abort</b><br>$\mathbf{z} = \mathbf{z}_k$<br>$(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$<br>$\mathbf{h} \leftarrow \text{MakeHint}(\mathbf{z}_2, \mathbf{w})$<br>$\sigma \leftarrow (\mathbf{z}_1, \mathbf{h})$ |                                                                                      |
|                                                                                                                                                                                                                                                                                                                                                                                                                | $\xrightarrow{\sigma}$ Reject if $\text{Verify}(\sigma, c) = 0$                      |

### CID simulator

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 : $\mathbf{z} \leftarrow \mathcal{D}_{\mathcal{R}, t, r}^{\ell+m+1}$<br>2 : $c \leftarrow \mathcal{U}(\mathcal{C})$<br>3 : $\mathbf{w} = \mathbf{A}\mathbf{z} - qc \pmod{2q}, w_0 \leftarrow LSB(\mathbf{w})$<br>4 : $\mathbf{z} = \mathbf{z}_k,$<br>$(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}, \mathbf{h} \leftarrow \text{MakeHint}(\mathbf{z}_2, \mathbf{w})$<br>5 : if $\ \mathbf{z}\ _2 > B$ , <b>return</b> $((\perp, c, (\mathbf{w}, w_0)))$<br>6 : <b>return</b> $((\mathbf{z}_1, \mathbf{h}), c, (\mathbf{w}, w_0))$ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 1: Canonical identification scheme and simulator corresponding to the unencoded version of our signature scheme.

## 4 Security Statement and Analysis

### 4.1 Theoretical Security

#### 4.1.1 CID

#### 4.1.2 error analysis

**Lemma 3** (A two-term Hölder-type inequality). *Let  $\alpha > 1$ , and let  $a, b, c, d \geq 0$ . Then*

$$\frac{(a+b)^\alpha}{(c+d)^{\alpha-1}} \leq \frac{a^\alpha}{c^{\alpha-1}} + \frac{b^\alpha}{d^{\alpha-1}}, \quad (1)$$

with the convention that a term of the form  $x^\alpha/0^{\alpha-1}$  is interpreted as  $+\infty$  when  $x > 0$ , and as 0 when  $x = 0$ .

*Proof.* The claim is trivial if  $c = 0$  or  $d = 0$ , so assume  $c, d > 0$ .

Define

$$u_1 = a c^{-\frac{\alpha-1}{\alpha}}, \quad u_2 = b d^{-\frac{\alpha-1}{\alpha}},$$

and

$$v_1 = c^{\frac{\alpha-1}{\alpha}}, \quad v_2 = d^{\frac{\alpha-1}{\alpha}}.$$

Then

$$u_1 v_1 = a, \quad u_2 v_2 = b,$$

and hence

$$a + b = u_1 v_1 + u_2 v_2.$$

Applying Hölder's inequality with conjugate exponents

$$p = \alpha, \quad q = \frac{\alpha}{\alpha-1},$$

we obtain

$$a + b \leq (u_1^\alpha + u_2^\alpha)^{1/\alpha} (v_1^q + v_2^q)^{1/q}.$$

Since

$$u_1^\alpha = \frac{a^\alpha}{c^{\alpha-1}}, \quad u_2^\alpha = \frac{b^\alpha}{d^{\alpha-1}},$$

and

$$v_1^q = c, \quad v_2^q = d,$$

this becomes

$$a + b \leq \left( \frac{a^\alpha}{c^{\alpha-1}} + \frac{b^\alpha}{d^{\alpha-1}} \right)^{1/\alpha} (c + d)^{(\alpha-1)/\alpha}.$$

Raising both sides to the power  $\alpha$  yields

$$(a + b)^\alpha \leq \left( \frac{a^\alpha}{c^{\alpha-1}} + \frac{b^\alpha}{d^{\alpha-1}} \right) (c + d)^{\alpha-1}.$$

Dividing by  $(c + d)^{\alpha-1}$  proves (1). □

**Theorem 2** (Rényi divergence bound). *Suppose the initial variable  $\mathbf{y}$  is distributed according to  $\rho_y$ , and let  $\rho_g$  be the target discrete Gaussian distribution. If the  $\alpha$ -order Rényi divergence between  $\rho_y$  and  $\rho_g$  is  $D_\alpha(\rho_y \parallel \rho_g)$ , then after  $\tau$  iterations, the distribution  $\rho_z$  of the output variable  $z$  satisfies, for every  $\alpha > 1$ ,*

$$D_\alpha(\rho_z \parallel \rho_g) \leq D_\alpha(\rho_y \parallel \rho_g) + \tau \cdot \log(1 + \epsilon_{rel}).$$

*Proof.* We prove the claim by induction on the number of iterations.

Assume that after  $j$  iterations, the intermediate variable  $z$  has distribution  $\rho_z$  satisfying

$$D_\alpha(\rho_z \| \rho_g) \leq D_\alpha(\rho_y \| \rho_g) + j \cdot \log(1 + \epsilon_{rel}).$$

We show that after one additional iteration, the new variable  $z'$  with distribution  $\rho_{z'}$  satisfies

$$D_\alpha(\rho_{z'} \| \rho_g) \leq D_\alpha(\rho_y \| \rho_g) + (j + 1) \cdot \log(1 + \epsilon_{rel}).$$

By definition of the  $\alpha$ -order Rényi divergence,

$$D_\alpha(\rho_{z'} \| \rho_g) = \frac{1}{\alpha - 1} \log \left( \sum_x \rho_{z'}(x)^\alpha \rho_g(x)^{1-\alpha} \right) \quad (2)$$

$$\begin{aligned} &= \frac{1}{\alpha - 1} \log \left( \sum_x \rho_{z'}(x)^\alpha ((\bar{f}_v \cdot \rho_g)(x + v) + (\bar{g}_v \cdot \rho_g)(x - v))^{1-\alpha} \right) \\ &\quad + \frac{1}{\alpha - 1} \log \left( \frac{\rho_g(x)^{1-\alpha}}{((\bar{f}_v \cdot \rho_g)(x + v) + (\bar{g}_v \cdot \rho_g)(x - v))^{1-\alpha}} \right). \end{aligned} \quad (3)$$

Using the relative error bound, we obtain

$$D_\alpha(\rho_{z'} \| \rho_g) \leq \frac{1}{\alpha - 1} \log \left( \sum_x \rho_{z'}(x)^\alpha ((\bar{f}_v \cdot \rho_g)(x + v) + (\bar{g}_v \cdot \rho_g)(x - v))^{1-\alpha} \right) + \log(1 + \epsilon_{rel}) \quad (4)$$

$$= \frac{1}{\alpha - 1} \log \left( \sum_x \frac{\rho_{z'}(x)^\alpha}{((\bar{f}_v \cdot \rho_g)(x + v) + (\bar{g}_v \cdot \rho_g)(x - v))^{\alpha-1}} \right) + \log(1 + \epsilon_{rel}). \quad (5)$$

Next, using the definition of  $\rho_{z'}$ , we rewrite the numerator as

$$\rho_{z'}(x) = (\bar{f}_v \cdot \rho_z)(x + v) + (\bar{g}_v \cdot \rho_z)(x - v).$$

Hence

$$D_\alpha(\rho_{z'} \| \rho_g) \leq \frac{1}{\alpha - 1} \log \left( \sum_x \frac{((\bar{f}_v \cdot \rho_z)(x + v) + (\bar{g}_v \cdot \rho_z)(x - v))^\alpha}{((\bar{f}_v \cdot \rho_g)(x + v) + (\bar{g}_v \cdot \rho_g)(x - v))^{\alpha-1}} \right) + \log(1 + \epsilon_{rel}). \quad (6)$$

Applying Lemma 3 pointwise, we obtain

$$\begin{aligned} &\sum_x \frac{((\bar{f}_v \cdot \rho_z)(x + v) + (\bar{g}_v \cdot \rho_z)(x - v))^\alpha}{((\bar{f}_v \cdot \rho_g)(x + v) + (\bar{g}_v \cdot \rho_g)(x - v))^{\alpha-1}} \\ &\leq \sum_x \left( \bar{f}_v(x) \frac{\rho_z(x)^\alpha}{\rho_g(x)^{\alpha-1}} + \bar{g}_v(x) \frac{\rho_z(x)^\alpha}{\rho_g(x)^{\alpha-1}} \right). \end{aligned} \quad (7)$$

Since our non-rejection scheme always outputs either  $x + v$  or  $x - v$ , the corresponding probabilities form a partition of unity. In particular, for every  $x$ , we have

$$\bar{f}_v(x) + \bar{g}_v(x) = 1.$$

Therefore,

$$\sum_x \left( \bar{f}_v(x) \frac{\rho_z(x)^\alpha}{\rho_g(x)^{\alpha-1}} + \bar{g}_v(x) \frac{\rho_z(x)^\alpha}{\rho_g(x)^{\alpha-1}} \right) = \sum_x \frac{\rho_z(x)^\alpha}{\rho_g(x)^{\alpha-1}}.$$

Therefore,

$$D_\alpha(\rho_{z'} \| \rho_g) \leq \frac{1}{\alpha - 1} \log \left( \sum_x \frac{\rho_z(x)^\alpha}{\rho_g(x)^{\alpha-1}} \right) + \log(1 + \epsilon_{rel}) \quad (8)$$

$$= D_\alpha(\rho_z \| \rho_g) + \log(1 + \epsilon_{rel}) \quad (9)$$

$$\leq D_\alpha(\rho_y \| \rho_g) + (j + 1) \cdot \log(1 + \epsilon_{rel}). \quad (10)$$

This completes the induction.  $\square$

### 4.1.3 UF-CMA and sUF-CMA Security

**Definition 1** (Rényi Honest-Verifier Zero-Knowledge). *Let  $\alpha > 1$ ,  $M \geq 0$ , and  $T \geq 0$ . A  $\Sigma$ -protocol is  $(M, T)$ -Rényi-HVZK of order  $\alpha$  if there exists a simulator  $\text{Sim}$  with runtime at most  $T$  such that, for every instance  $x$ , if  $(w', c', z')$  denotes an honestly generated transcript and  $(w, c, z)$  denotes the output of  $\text{Sim}(x)$ , then*

$$D_\alpha((w', c', z') \parallel (w, c, z)) \leq M.$$

*This includes aborting transcripts, i.e., those for which  $z = \perp$ .*

**Theorem 3.** *(adapted from) Let  $M, \alpha, T_{\text{Sim}} \geq 0$ ,  $B \geq 0$ , and let  $H$  be a hash function modeled as a random oracle. Assume that*

$$\Sigma = ((P_1, P_2), (V_1, V_2))$$

*is an  $(M, T_{\text{Sim}})$ -Rényi-HVZK of order  $\alpha$  a public-coin identification protocol, and that the commitment message of the prover has min-entropy  $\alpha$ . Let  $\mathcal{A}$  be any adversary against the UF-CMA security of*

$$\text{SIG}_B = \text{FSB}[\Sigma, H]$$

*that makes at most  $Q_H$  queries to the random oracle  $H$  and  $Q_S$  classical queries to the signing oracle. Consider a scenario in which we have an array (a data structure) of  $N$  classical strings  $x_1, \dots, x_N$ . Quantum Random Access Classical Memory (QRACM) allows us to load this data into a quantum register in superposition. More precisely, a QRACM operation is defined by*

$$U_{\text{QRACM}} : |i\rangle |y_i\rangle \mapsto |i\rangle |y_i \oplus x_i\rangle$$

*for all indices  $i$  and all  $y_i$ , and is extended to general quantum states by linearity. Then there exists a quantum adversary  $\mathcal{B}$  against the UF-NMA security of  $\text{SIG}_B$  such that:*

- *In the ROM, algorithm  $\mathcal{B}$  runs in time*

$$\text{Time}(\mathcal{A}) + O((T_{\text{Sim}} \cdot B \cdot Q_S + Q_H) \log(B \cdot Q_S + Q_H)),$$

$$\text{and } \text{Adv}_{\text{SIG}_B}^{\text{CMA}}(\mathcal{A}) \leq M^{B \cdot Q_S} \cdot (\text{Adv}_{\text{SIG}_B}^{\text{UF-NMA}}(\mathcal{B}))^{(a-1)/a} + 2^{-\alpha} \cdot B \cdot Q_S \cdot (B \cdot Q_S + Q_H + 1)$$

- *In the QROM, algorithm  $\mathcal{B}$  runs in time*

$$\text{Time}(\mathcal{A}) + O((T_{\text{Sim}} \cdot B \cdot Q_S + Q_H) \log(B \cdot Q_S))$$

*with QRACM, and*

$$\text{Time}(\mathcal{A}) + O((T_{\text{Sim}} \cdot B \cdot Q_S + Q_H) \cdot (B \cdot Q_S))$$

*without QRACM, and*

$$\text{Adv}_{\text{SIG}_B}^{\text{CMA}}(\mathcal{A}) \leq M^{B \cdot Q_S} \cdot (\text{Adv}_{\text{SIG}_B}^{\text{UF-NMA}}(\mathcal{B}))^{(a-1)/a} + 2^{-\alpha/2} \cdot 3B \cdot Q_S \cdot \sqrt{B \cdot Q_S + Q_H + 1}$$

*Proof.* The proof is similar to the one of Theorem 10[cite].

**Game  $G_0$ .** Same as in Theorem 10, The first game is the UF-CMA security game.

**Game  $G_1$ .** In this game, the challenges of the transcripts are no longer computed by the random oracle, but are instead sampled independently and uniformly each time. Then, the random oracle is reprogrammed according to the new challenges.

To bound the distance between Game  $G_0$  and Game  $G_1$ , we construct a wrapper  $\mathcal{D}$  around  $\mathcal{A}$  that uses  $\mathcal{A}$  to solve a reprogramming game. It works as in Figure 16. Note that if  $b = 0$  in Figure 16, then  $\mathcal{D}$  perfectly simulates  $G_0$ , and otherwise it perfectly simulates  $G_1$ . Therefore,

$$|\Pr[1 \leftarrow G_0^{\mathcal{A}}] - \Pr[1 \leftarrow G_1^{\mathcal{A}}]| \leq |\Pr[1 \leftarrow \text{Reprogram}_0^{\mathcal{D}}] - \Pr[1 \leftarrow \text{Reprogram}_1^{\mathcal{D}}]|.$$

**Game:**

```

1:  $\mathcal{M} := \emptyset$ 
2:  $(vk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 
3:  $(\mu^*, \sigma^*) \leftarrow \mathcal{A}^{H, \text{Sign}(sk, \cdot)}(vk)$ 
4: Parse  $\sigma^* = (w^*, z^*)$ 
5:  $c^* := H(w^* \parallel \mu^*)$ 
6: return  $\mu^* \notin \mathcal{M} \wedge V_2(vk, w^*, c^*, z^*)$ 

```

**Sign( $sk, \mu$ ):**

```

1:  $\mathcal{M} := \mathcal{M} \cup \{\mu\}$ 
2:  $(w, c, z) \leftarrow \text{GetTrans}(\mu)$ 
3: if  $z = \perp$  return  $\perp$ 
4: return  $\sigma = (w, z)$ 

```

**GetTrans( $\mu$ ):**

```

1:  $\kappa := 0$ 
2: while  $z = \perp$  and  $\kappa \leq B$ 
3:    $(w, st) \leftarrow P_1(sk)$ 
4:    $c := H(w \parallel \mu)$ 
5:    $z \leftarrow P_2(sk, w, c, st)$ 
6:    $\kappa := \kappa + 1$ 
7: return  $(w, c, z)$ 

```

Figure 2: UF-CMA security game(game  $G_0$ )

During the game, the distinguisher  $\mathcal{D}$  makes  $B \cdot Q_S$  reprogramming queries and  $B \cdot Q_S + Q_H + 1$  random oracle queries. In the ROM, the advantage of  $\mathcal{D}$  is bounded by

$$B \cdot Q_S \cdot (B \cdot Q_S + Q_H + 1) 2^{-\alpha}.$$

In the QROM, it follows that the advantage of  $\mathcal{D}$  is bounded by

$$3B \cdot Q_S^2 \cdot \sqrt{(B \cdot Q_S + Q_H + 1) 2^{-\alpha}}.$$

**Game :**

```

1:  $\mathcal{M} := \emptyset$ 
2:  $(vk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 
3:  $(\mu^*, \sigma^*) \leftarrow \mathcal{A}^{H, \text{Sign}(sk, \cdot)}(vk)$ 
4: Parse  $\sigma^* = (w^*, z^*)$ 
5:  $c^* := H(w^* \parallel \mu^*)$ 
6: return  $\mu^* \notin \mathcal{M} \wedge V_2(vk, w^*, c^*, z^*)$ 

```

**Sign( $sk, \mu$ ) :**

```

1:  $\mathcal{M} := \mathcal{M} \cup \{\mu\}$ 
2:  $(w, c, z) \leftarrow \text{GetTrans}(\mu)$ 
3: if  $z = \perp$  return  $\perp$ 
4: return  $\sigma = (w, z)$ 

```

**GetTrans( $\mu$ ) :**

```

1:  $\kappa := 0$ 
2: while  $z = \perp$  and  $\kappa \leq B$ 
3:    $(w, st) \leftarrow P_1(sk)$ 
4:    $c \leftarrow U(\mathcal{C})$ 
5:    $z \leftarrow P_2(sk, w, c, st)$ 
6:    $H = H'(w \parallel \mu \mapsto c)$ 
7:    $\kappa := \kappa + 1$ 
8: return  $(w, c, z)$ 

```

Figure 3: Game  $G_1$ . The difference from  $G_0$  is highlighted in blue.

**Game  $G_2$ .** Let  $\text{Sim}$  be the zero-knowledge simulator for  $\Sigma$ . In this game we modify  $\text{GET-TRANS}$  such that the transcripts are now produced by  $\text{Sim}$  and without the secret key. By the

**Game :**

```

1:  $\mathcal{M} := \emptyset$ 
2:  $(vk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 
3:  $(\mu^*, \sigma^*) \leftarrow \mathcal{A}^{H, \text{Sign}(sk, \cdot)}(vk)$ 
4: Parse  $\sigma^* = (w^*, z^*)$ 
5:  $c^* := H(w^* \parallel \mu^*)$ 
6: return  $\mu^* \notin \mathcal{M} \wedge V_2(vk, w^*, c^*, z^*)$ 

```

**Sign( $sk, \mu$ ) :**

```

1:  $\mathcal{M} := \mathcal{M} \cup \{\mu\}$ 
2:  $(w, c, z) \leftarrow \text{GetTrans}(\mu)$ 
3: if  $z = \perp$  return  $\perp$ 
4: return  $\sigma = (w, z)$ 

```

**GetTrans( $\mu$ ) :**

```

1:  $\kappa := 0$ 
2: while  $z = \perp$  and  $\kappa \leq B$ 
3:    $c \leftarrow U(\mathcal{C})$ 
4:    $(w, z) \leftarrow \text{Sim}(vk, c)$ 
5:    $H := H^{w \parallel \mu \rightarrow c}$ 
6:    $\kappa := \kappa + 1$ 
7: return  $(w, c, z)$ 

```

Figure 4: Game  $G_2$ . The difference from  $G_1$  is highlighted in blue.

statistical HVZK property of the  $\Sigma$ -protocol, it follows that

$$\Pr[1 \leftarrow G_1^A] \leq M^{B \cdot Q_S} \cdot (\Pr[1 \leftarrow G_2^A])^{(a-1)/a}$$

**Game  $G_3$ , Game  $G_4$  and Game  $G_5$ .** the same as  $\square$ .

Combining the differences between each game, we obtain the final inequality.  $\square$

For the reduction proof of strong unforgeability, our analysis differs substantially from that in the reference. In the reference, because the statistical error is symmetric, one can estimate the probability of event 2 in game 1 in order to bound the probability of event 2 in game 2. The setting considered here is different, since Rényi divergence is not symmetric. Therefore, we introduce a new notion, called a verifiable simulator. We prove that the simulator of our algorithm satisfies this property, and then carry out the analysis based on it.

**Definition 2** ( $\beta$ -Verifiable Simulator). *Let  $\Pi = (P, V)$  be an identification protocol. A probabilistic polynomial-time algorithm  $\text{Sim}$  is called a  $\beta$ -verifiable simulator if, on input a statement  $x$ , it outputs a transcript  $(w, c, z)$  such that*

$$\Pr[z = \perp] \leq \beta,$$

*and for all outputs with  $z \neq \perp$ , the transcript is accepting for the verifier, i.e.,*

$$V(x, w, c, z) = 1.$$

**Definition 3** (Simulator Commitment Min-Entropy). *For  $\alpha \geq 0$ , we say that an identification scheme  $\Sigma = ((P_1, P_2), (V_1, V_2))$  with instance generator  $\text{Gen}$  has simulator commitment min-entropy  $\alpha$  if*

$$H_\infty[w \mid (w, \text{st}) \leftarrow \text{Sim}_1(x)] \geq \alpha,$$

*for all  $(x, y) \leftarrow \text{Gen}(1^\lambda)$ , where  $\text{Sim}_1$  denotes the first-message simulator of the honest-verifier zero-knowledge simulator.*

**Theorem 4.** (adpted form [cite]) *Let  $M, \alpha, \beta, \gamma, T_{\text{Sim}} \geq 0$ ,  $B \geq 0$ , and  $H$  a hash function modeled as a random oracle. Assume that  $\Sigma = ((P_1, P_2), (V_1, V_2))$  is an  $(M, T_{\text{Sim}})$ -Rényi-HVZK and*

$(\gamma, \beta)$ -correct public-coin identification protocol, and that its zero-knowledge simulator is also  $\beta$ -verifiable and has min-entropy  $\alpha$ , and that the commitment message of the prover has min-entropy  $\alpha$ . Let  $A$  be any arbitrary adversary against  $sUF$ -CMA security of  $SIG_B = FSB[\Sigma, H]$  that issues at most  $Q_H$  queries to the random oracle  $H$  and  $Q_S$  classical queries to the signing oracle. There exist quantum adversaries  $B$  and  $C$  respectively against  $UF$ -NMA security of  $SIG_B$  and the computational unique response property of  $\Sigma$  as follows:

- In the ROM, both  $B$  and  $C$  have runtime

$$\text{Time}(A) + O((T_{\text{Sim}} \cdot B \cdot Q_S + Q_H) \log(B \cdot Q_S + Q_H)),$$

and

$$\begin{aligned} \text{Adv}_{\text{SIG}_B}^{\text{CMA}}(\mathcal{A}) &\leq M^{B \cdot Q_S} \cdot (\text{Adv}_{\text{SIG}_B}^{\text{UF-NMA}}(\mathcal{B}) + \text{Adv}_{\Sigma}^{\text{CUR}}(\mathcal{C}) + Q_S(\beta^B + B \cdot 2^{-\alpha}))^{(a-1)/a} \\ &\quad + 2^{-\alpha} \cdot B \cdot Q_S \cdot (B \cdot Q_S + Q_H + 1). \end{aligned}$$

- In the QROM, the runtimes of  $B$  and  $C$  are

$$\text{Time}(A) + O((T_{\text{Sim}} \cdot B \cdot Q_S + Q_H) \log(B \cdot Q_S))$$

with QRACM, and

$$\text{Time}(A) + O((T_{\text{Sim}} \cdot B \cdot Q_S + Q_H) \cdot (B \cdot Q_S))$$

without QRACM, and

$$\begin{aligned} \text{Adv}_{\text{SIG}_B}^{\text{CMA}}(\mathcal{A}) &\leq M^{B \cdot Q_S} \cdot (\text{Adv}_{\text{SIG}_B}^{\text{UF-NMA}}(\mathcal{B}) + \text{Adv}_{\Sigma}^{\text{CUR}}(\mathcal{C}) + Q_S(\beta^B + B \cdot 2^{-\alpha}))^{(a-1)/a} \\ &\quad + 2^{-\alpha/2} \cdot 3B \cdot Q_S \cdot \sqrt{B \cdot Q_S + Q_H + 1}. \end{aligned}$$

*Proof.* The proof is similar to the one of Theorem 11[cite].

All the games remain consistent with those in Theorem 11 of [?]. The only difference lies in our analysis of the probability of `event2` in Game 2. Since our simulator is perfectly accepting in all non-aborting executions, `event2` in Game 2 can occur in only two cases: (i) the simulator outputs an aborting transcript; or (ii) the commitment  $w^*$  does not appear in the final transcript but instead appears during the intermediate iterations. The probability of the latter event is bounded by the high-entropy guarantee of the simulator. we have

$$\Pr[\text{Event2} \mid G_2^A] \leq Q_S(\beta^B + B \cdot 2^{-\alpha}). \quad (11)$$

□

**Lemma 4** (Data Processing for Rényi Divergence). *Let  $(\Omega, \mathcal{F})$  and  $(\Omega', \mathcal{F}')$  be measurable spaces, and let  $P, Q$  be two probability measures on  $(\Omega, \mathcal{F})$ . Let  $f : \Omega \rightarrow \Omega'$  be a measurable function, and denote by  $f(P)$  and  $f(Q)$  the pushforward measures induced by  $f$ , i.e.,*

$$f(P)(A) := P(f^{-1}(A)), \quad f(Q)(A) := Q(f^{-1}(A)) \quad \text{for all } A \in \mathcal{F}.$$

*Let  $\alpha > 1$ . If the Rényi divergence of order  $\alpha$  satisfies*

$$D_\alpha(P \parallel Q) \leq M,$$

*then*

$$D_\alpha(f(P) \parallel f(Q)) \leq M.$$

*Proof.* Let  $L = \frac{dP}{dQ}$  be the Radon-Nikodym derivative of  $P$  with respect to  $Q$  on the measurable space  $(\Omega, \mathcal{F})$ . Since  $D_\alpha(P\|Q) \leq M < \infty$ , such a derivative exists and  $P \ll Q$ . For the pushforward measures, the condition  $P \ll Q$  implies  $f(P) \ll f(Q)$ , so there exists a Radon-Nikodym derivative  $\tilde{L} = \frac{df(P)}{df(Q)}$  on  $(\Omega', \mathcal{F}')$ .

By the definition of pushforward measures and the change of variables formula, for any  $A \in \mathcal{F}'$ , we have:

$$\int_A \tilde{L} df(Q) = f(P)(A) = P(f^{-1}(A)) = \int_{f^{-1}(A)} L dQ.$$

Let  $\sigma(f) = \{f^{-1}(A) : A \in \mathcal{F}'\}$  be the  $\sigma$ -algebra generated by  $f$ . The identity above implies that  $\tilde{L} \circ f$  is a version of the conditional expectation of  $L$  given  $\sigma(f)$  under measure  $Q$ , i.e.,

$$\tilde{L} \circ f = \mathbb{E}_Q[L|\sigma(f)], \quad Q\text{-a.s.}$$

The divergence is defined as  $D_\alpha(P\|Q) = \mathbb{E}_Q[L^\alpha]$ . For the transformed measures, we have:

$$D_\alpha(f(P)\|f(Q)) = \int_{\Omega'} \tilde{L}(y)^\alpha df(Q)(y) = \int_\Omega (\tilde{L} \circ f)^\alpha dQ = \mathbb{E}_Q[(\tilde{L} \circ f)^\alpha].$$

Substituting the conditional expectation form, we get:

$$D_\alpha(f(P)\|f(Q)) = \mathbb{E}_Q[(\mathbb{E}_Q[L|\sigma(f)])^\alpha].$$

Since  $\alpha > 1$ , the function  $\phi(x) = x^\alpha$  is convex for  $x \geq 0$ . By Jensen's inequality for conditional expectations, we have:

$$(\mathbb{E}_Q[L|\sigma(f)])^\alpha \leq \mathbb{E}_Q[L^\alpha|\sigma(f)], \quad Q\text{-a.s.}$$

Taking the expectation with respect to  $Q$  on both sides and applying the law of total expectation:

$$\mathbb{E}_Q[(\mathbb{E}_Q[L|\sigma(f)])^\alpha] \leq \mathbb{E}_Q[\mathbb{E}_Q[L^\alpha|\sigma(f)]] = \mathbb{E}_Q[L^\alpha].$$

This simplifies to:

$$D_\alpha(f(P)\|f(Q)) \leq D_\alpha(P\|Q).$$

Given that  $D_\alpha(P\|Q) \leq M$ , we conclude  $D_\alpha(f(P)\|f(Q)) \leq M$ .  $\square$

**Commitment min-entropy.** we note that the commitment min-entropy of the compressed scheme is easily lower-bounded by close to  $n$  bits. This is the case as  $\mathbf{w}_0 = \text{LSB}(\mathbf{y}_0)_j$  is part of the commitment, where  $\mathbf{y}_0$  is sampled from a discrete Gaussian distribution with a relatively large standard deviation. As such, the coefficients of the first element of  $\mathbf{w}_0$  are statistically close to uniformly random, and  $\mathbf{w}_0$  therefore has a min-entropy that is not significantly smaller than  $n$ .

Note, however, that this should significantly underestimate the commitment min-entropy of the scheme, as it only accounts for the min-entropy in  $\mathbf{w}_0$ . Although we do not prove this, similarly to in HAETAE [11], we feel that it is safe to assume that the actual commitment min-entropy is significantly larger than this. We therefore ignore terms which depend on the min-entropy when we use Theorem 2. Furthermore, we note that the uncompressed variant of the scheme obviously has at least as high min-entropy as the compressed variant.

**Divergence and verifiable, min entropy HVZK.** Apply lemma we know  $R_a(w', c', z') \parallel (w, c, z)$  is less than  $2^{-80}$ , under our parameter. The verifiability is obvious. we obtain min entropy by the same argument above.

**CUR** The adversary  $\mathcal{A}$  would instead produce  $w_0, c, z_{a,1}, h_a, z_{b,1}, h_b$ , with  $(z_{a,1}, h_a) \neq (z_{b,1}, h_b)$ . Assuming that this guarantees that the reconstructed  $z'_a$  and  $z'_b$  differ, As the reconstructed vector  $z'$  is given by

$$z' = \begin{bmatrix} z_1^T, & z_2'^T \end{bmatrix}^T,$$

the equality  $z'_a = z'_b$  implies that  $z_{a,1} = z_{b,1}$ . Additionally, given  $z_{a,1} = z_{b,1}$ , in order for  $z_{a,2} = z_{b,2}$  to hold, it is required that

$$w_{0,a} \equiv w_{0,b} \pmod{2(q-1)},$$

which is only possible if

$$h_a \equiv h_b \pmod{2(q-1)}.$$

To ensure computational uniqueness of the response, it is therefore necessary to verify that  $h$  is the canonical representative modulo  $2(q-1)$ . With such a verification, it is guaranteed that two distinct accepting transcripts for the compressed CID scheme imply two distinct reconstructed vectors  $z'_a$  and  $z'_b$ , both of which have norm at most  $B_v$ . this directly yields a solution to the MSIS variant with bound  $2B_v$ .

#### 4.1.4 UF-NMA Security

**Definition 4** (MLWE $_{q,m,\ell,\mathcal{D}}$  Problem). *For integers  $m, \ell$ , and a probability distribution  $\mathcal{D}$ , we say that the advantage of algorithm  $\mathcal{A}$  in solving the decisional MLWE $_{q,m,\ell,\mathcal{D}}$  problem is*

$$\begin{aligned} \text{Adv}_{q,m,\ell,\mathcal{D}}^{\text{MLWE}} := & \left| \Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m \times \ell}; \mathbf{t} \leftarrow \mathcal{R}_q^m; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t})] \right. \\ & \left. - \Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m \times \ell}; \mathbf{s}_1 \leftarrow \mathcal{D}^\ell; \mathbf{s}_2 \leftarrow \mathcal{D}^m; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2)] \right|. \end{aligned}$$

This **SelfTargetMSIS** problem is parametrized by a size bound  $B_v > 0$  and integers  $q, m, \ell > 0$ . Furthermore, it depends on a hash function  $H$  from  $\mathcal{R}_{2q}^m \times \mathcal{M}$  to  $\mathcal{C}$  where  $\mathcal{M} \subseteq \{0, 1\}^*$  is a message space and  $\mathcal{C} \subseteq \mathcal{R}_2$  is a challenge space.

**Definition 5** (SelfTargetMSIS $_{H,q,m,\ell,B_v}$  problem). *Given  $\mathbf{A}_0 \in \mathcal{R}_q^{m \times \ell}$  and  $\mathbf{b} \in \mathcal{R}_q^m$  with elements sampled uniformly at random in  $\mathcal{R}_q$ , an adversary  $\mathcal{A}$  against the SelfTargetMSIS $_{H,q,m,\ell,B_v}$  problem is to produce  $(\mathbf{z}, c, \mu) \in (\mathcal{R}^{m+\ell+1}, \mathcal{C}, \mathcal{M})$  such that  $\mathbf{z} \neq \mathbf{0}$ ,  $\|\mathbf{z}\|_2 < B_v$  and*

$$H(\mathbf{A}\mathbf{z} - q\mathbf{c}\mathbf{j} \pmod{2q}, \mu) = c$$

where  $\mathbf{A} = [q\mathbf{j} - 2\mathbf{b}, 2\mathbf{A}_0, 2\mathbf{I}] \pmod{2q}$ .

The UF-NMA security of the uncompressed version of our scheme is almost directly given by the assumed hardnesses of the decisional MLWE problem and the SelfTargetMSIS problem. For the compressed version of our scheme, the UF-NMA security similarly relies on the assumed hardness of these problems, but not quite as obviously. For both versions of our scheme, we argue for their UF-NMA security by the following lemma.

**Lemma 5.** *Let  $\mathcal{A}$  be an adversary against the UF-NMA security of our signature scheme. Then, there exist two adversaries  $\mathcal{B}$  and  $\mathcal{B}'$ , each using  $\mathcal{A}$  once and performing negligible additional work, such that*

$$\text{Adv}^{\text{UF-NMA}}(\mathcal{A}) \leq 1/\delta \cdot \text{Adv}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}^{\text{MLWE}}(\mathcal{B}) + \text{Adv}_{H,q,m,\ell,B_v}^{\text{SelfTargetMSIS}}(\mathcal{B}'),$$

where  $\delta$  is the probability of an accepted secret key occurring.

*Proof.* We define two games with  $\text{Game}_0, \text{Game}_1$ .  $\text{Game}_0$  represents the UF-NMA security experiment for our scheme. The advantage of an adversary  $\mathcal{A}$  in  $\text{Game}_0$  corresponds directly to the advantage  $\text{Adv}^{\text{UF-NMA}}(\mathcal{A})$  against the real scheme. For  $\text{Game}_1$ ,  $\mathbf{A}_0$  and  $\mathbf{b}$  in the public key are uniformly random instead of given by an MLWE instance. The advantage of the adversary  $\mathcal{A}$  in  $\text{Game}_1$  is defined as  $G_1$ .

For the proof, we consider that if  $\mathcal{A}$  has different advantages against the two games, this can be used to solve the decisional MLWE problem. Formally, the adversary  $\mathcal{B}$  is given an instance

$(\mathbf{A}_0, \mathbf{b})$ .  $\mathcal{B}$  sets  $(\mathbf{A}_0, \mathbf{b})$  as the verification key of our scheme for  $\mathcal{A}$ , with the probability  $\delta$ . The probability  $\delta$  exists due to the key rejection. Under the condition that  $(\mathbf{A}, \mathbf{b})$  is successfully embedded, if a forgery is created by  $\mathcal{A}$  under  $(\mathbf{A}, \mathbf{b})$ , then  $\mathcal{B}$  outputs 1; otherwise, it outputs 0. In the case where  $(\mathbf{A}, \mathbf{b})$  is not successfully embedded,  $\mathcal{B}$  outputs 1 with probability  $1/2$ .

Next, we analyze the advantage of  $\mathcal{B}$  in solving the decisional MLWE problem. If  $(\mathbf{A}, \mathbf{b})$  is an MLWE instance, the probability that  $\mathcal{B}$  outputs 1 is  $\delta \cdot \text{Adv}^{\text{UF-NMA}}(\mathcal{A}) + (1 - \delta) \cdot 1/2$ . If  $(\mathbf{A}, \mathbf{b})$  is sampled uniformly at random, the probability that  $\mathcal{B}$  outputs 1 is  $\delta \cdot G_1 + (1 - \delta) \cdot 1/2$ . Therefore, we conclude that the advantage of  $\mathcal{B}$  in solving the MLWE problem is

$$\text{Adv}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}^{\text{MLWE}}(\mathcal{B}) = \delta \cdot |\text{Adv}^{\text{UF-NMA}}(\mathcal{A}) - G_1|.$$

The algorithm  $\mathcal{B}'$  instead relies on whatever advantage  $\mathcal{A}$  has in  $\text{Game}_1$ . To achieve this, the input to the target SelfTargetMSIS problem is used to construct the verification key that is provided to  $\mathcal{A}$ . This verification key is thus constructed from uniformly random  $\mathbf{A}_0$  and  $\mathbf{b}$ . For the uncompressed version of our signature scheme, a forgery produced by  $\mathcal{A}$  directly corresponds to a solution for the SelfTargetMSIS problem of  $\mathcal{B}'$ , which proves the statement for the uncompressed version.

For the compressed version of our signature scheme, the random oracle that  $\mathcal{B}'$  provides when simulating  $\mathcal{A}$  is given by

$$\mathbf{H}'(\mathbf{w}', w_0, \mu) = \mathbf{H}(\mathbf{w}' + w_0, \mu)$$

where  $\mathbf{H}$  is the random oracle for the SelfTargetMSIS instance. A forgery from the signature scheme consists of a signature  $(\mathbf{z}_1, \mathbf{h}, c)$  and a message  $\mu$  such that

$$\mathbf{H}'(\mathbf{w}', w_0, \mu) = \mathbf{H}(\mathbf{w}' + w_0, \mu) = c$$

for  $\mathbf{w}'$  and  $w_0$  defined as in  $\text{Verify}((\mathbf{z}_1, \mathbf{h}, c), \mathbf{A}, \mu)$ . A valid forgery additionally corresponds to a vector  $\mathbf{z}' = [\mathbf{z}_1^T, \mathbf{z}_2^T]^T$  such that  $\|\mathbf{z}'\|_2 \leq B_v$ , where  $\mathbf{z}_2' = (\mathbf{w}' + w_0 - \tilde{\mathbf{w}}')/2$ . Given such a forgery, we claim that  $(\mathbf{z}', c, \mu)$  solves the target SelfTargetMSIS instance. This is the case as

$$\mathbf{A}\mathbf{z}' = \mathbf{A}_1\mathbf{z}_1 + 2\mathbf{z}_2' = \mathbf{w}' + w_0 + qc\mathbf{j}$$

and thus

$$\mathbf{H}(\mathbf{A}\mathbf{z}' - qc\mathbf{j}, \mu) = \mathbf{H}(\mathbf{w}' + w_0, \mu) = c$$

as claimed. Furthermore, valid signatures satisfy that  $\|\mathbf{z}'\|_2 \leq B_v$  and therefore the norm bound for the SelfTargetMSIS instance is satisfied. That is  $G_1 \leq \text{Adv}_{\mathbf{H},q,m,\ell,B_v}^{\text{SelfTargetMSIS}}(\mathcal{B}')$ .  $\square$

## 4.2 Concrete Security Estimates

Provide the exact bit-security against known classical and quantum attacks (e.g., Core-SVP hardness).

## 4.3 Side-Channel Resistance and Implementation Security

Discuss resistance to side-channel attacks.

## 4.4 Other Security Properties

Discuss Perfect Forward Secrecy (PFS), and behavior under key reuse (ephemeral nature of the KE).

## 5 Performance Evaluation and Implementation Analysis

This section presents a comprehensive evaluation of the proposed scheme’s performance across different software platforms. We analyze the computational complexity, storage requirements, and communication overhead, demonstrating the algorithm’s efficiency for both high-performance desktop processors and resource-constrained embedded devices.

### 5.1 Evaluation Methodology and Platforms

To ensure reproducibility and demonstrate the scheme’s versatility, we define the benchmarking environments as follows:

- **High-Performance Platform (x86\_64):** Intel Core i7-1185G7 (Tiger Lake) @ 3.0GHz, 16GB RAM.
  - *Compiler:* `gcc 11.2`
  - *Flags:* `-O3 -fomit-frame-pointer -march=native -mavx2`
- **Embedded Platform (32-bit ARM):** STM32F407 Discovery Board (ARM Cortex-M4) @ 168MHz, 192KB SRAM, 1MB Flash.
  - *Compiler:* `arm-none-eabi-gcc 10.3`
  - *Flags:* `-O3 -mthumb -mcpu=cortex-m4 -mfloat-abi=hard -mfpu=fpv4-sp-d16`
- **Metric Definition:** Performance is measured in CPU cycles (median of 10,000 runs). For the embedded platform, we also report stack usage to demonstrate suitability for memory-constrained environments.

### 5.2 Software Performance on x86 Processors

We provide two implementations for the x86 platform: a portable ISO C99 reference implementation and an optimized implementation utilizing AVX2 instruction set extensions.

#### 5.2.1 Benchmarking Results (Intel Core i7)

Table 5 compares the cycle counts for the three protocol stages: Key Generation (KeyGen), Encapsulation (Encaps), and Decapsulation (Decaps).

| Table 5: Performance (CPU Cycles) on Intel Core i7 (Tiger Lake) |                  |               |               |               |
|-----------------------------------------------------------------|------------------|---------------|---------------|---------------|
| Parameter Set                                                   | Implementation   | KeyGen        | Encaps        | Decaps        |
| <b>Algo-Level1</b>                                              | Reference (C99)  | 120,000       | 145,000       | 160,000       |
|                                                                 | Optimized (AVX2) | <b>15,000</b> | <b>18,000</b> | <b>21,000</b> |
| <b>Algo-Level3</b>                                              | Reference (C99)  | 250,000       | 290,000       | 310,000       |
|                                                                 | Optimized (AVX2) | <b>32,000</b> | <b>38,000</b> | <b>42,000</b> |

The AVX2 optimization achieves a speedup factor of approximately  $6\times$  to  $8\times$  compared to the reference implementation, primarily due to the parallelization of polynomial arithmetic using 256-bit vector registers.

### 5.3 Performance on Embedded Systems (ARM Cortex-M4)

To verify the applicability of the proposed scheme in IoT scenarios, we evaluated the performance on a 32-bit ARM Cortex-M4 microcontroller.

#### 5.3.1 Benchmarking Results (STM32F4)

Table 6 details the cycle counts and stack usage. The implementation is optimized for 32-bit arithmetic but remains written in C for portability.

Table 6: Performance on ARM Cortex-M4 (STM32F407 @ 168MHz)

| Parameter Set      | CPU Cycles ( $\times 10^3$ ) |        |        | Stack Usage<br>(Bytes) |
|--------------------|------------------------------|--------|--------|------------------------|
|                    | KeyGen                       | Encaps | Decaps |                        |
| <b>Algo-Level1</b> | 850                          | 920    | 1,050  | 4,096                  |
| <b>Algo-Level3</b> | 1,600                        | 1,850  | 2,100  | 6,144                  |

The results demonstrate that the scheme can execute KeyGen, Encaps, and Decaps operations within reasonable timeframes (under 10ms for Level 1 at 168MHz) while maintaining a low memory footprint suitable for embedded devices.

### 5.4 Optimization Techniques

To achieve the performance metrics reported above, we employed specific optimization strategies targeting the characteristics of each platform.

#### 5.4.1 Vectorization with AVX2 (x86)

Our high-performance implementation leverages the 256-bit wide registers of the AVX2 instruction set:

- **Parallel NTT:** We utilize a vectorized Number Theoretic Transform (NTT) where 16 coefficients (16-bit) are processed simultaneously in a single AVX2 register.
- **Vectorized Montgomery Reduction:** We implement modular multiplication using `vpnullw` and `vpmulhw` instructions to perform reduction without expensive division operations.
- **Register Allocation:** To minimize memory access latency, we carefully schedule the NTT layers to keep intermediate results within the CPU registers as long as possible (lazy reduction).

#### 5.4.2 Optimizations for 32-bit Embedded Systems

For the ARM Cortex-M4 implementation, we focused on efficient 32-bit arithmetic and memory management:

- **32-bit Friendly Modulo Arithmetic:** Since the Cortex-M4 lacks 64-bit vector instructions, we optimized the modular reduction (Barrett/Montgomery) to operate efficiently within 32-bit general-purpose registers, avoiding unnecessary 64-bit promotion where possible.

- **Stack Optimization:** We minimized recursive calls and reused memory buffers for intermediate polynomial calculations to ensure the stack usage remains within the limits of typical microcontrollers (e.g.,  $\leq 8\text{KB}$ ).
- **Flash Memory Alignment:** Constant polynomials and tables are aligned to flash memory page boundaries to utilize the MCU’s pre-fetch buffer efficiently.

## 5.5 Transmission and Storage Overhead

Minimizing bandwidth is critical for network protocols. Table 7 presents the sizes of the public key ( $pk$ ) and ciphertext ( $ct$ ).

Table 7: Communication Sizes and Protocol Compatibility

| Parameter Set | $ pk $ (Bytes) | $ ct $ (Bytes) | Total (Bytes) | Fits in Ethernet MTU?     |
|---------------|----------------|----------------|---------------|---------------------------|
| Algo-Level1   | 800            | 768            | 1568          | <b>Yes (Jumbo)</b> / Frag |
| Algo-Level3   | 1184           | 1088           | 2272          | Requires Frag             |

Our scheme (Level 1) is specifically optimized to fit within standard MTU limits when possible, significantly reducing the probability of packet fragmentation and loss in UDP-based protocols.

## 6 Features Statement

### 6.1 Innovation

Highlight the "Dancing with Imperfection" philosophy: trading minor decryption failures for significant bandwidth gains via the IND-CPAF framework.

### 6.2 Compatibility and Real-World Applicability

Explain how this Key Exchange protocol perfectly integrates with existing IKEv2 state machines. By keeping the KE payload under the UDP MTU, it avoids IP fragmentation and eliminates the need for the `IKE_INTERMEDIATE` exchange, saving a full Round Trip Time (RTT).

### 6.3 Flexibility and Scalability

Discuss how the protocol can be paired with standard signature schemes (e.g., SM2 or future PQC signatures) in the `IKE_AUTH` phase, maintaining the established dual-certificate PKI architecture.

## References

- [DDL13] Léo Ducas, Alain Durmus, Tancrede Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal gaussians. In *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 40–56. Springer, 2013.
- [Gär25] Joel Gärtner. Compact lattice signatures via iterative rejection sampling. In *Annual International Cryptology Conference*, pages 514–547. Springer, 2025.
- [HPRR20] James Howe, Thomas Präst, Thomas Ricosset, and Mélissa Rossi. Isochronous Gaussian sampling: From inception to implementation. In *Post-Quantum Cryptography – PQCrypto 2020*, volume 12100 of *Lecture Notes in Computer Science*, pages 53–71. Springer, 2020.
- [Ros20] Mélissa Rossi. *Extended Security of Lattice-Based Cryptography*. PhD thesis, Paris Sciences et Lettres University, France, 2020.

## A Intellectual Property Statement

## B Test Vectors

This section provides a sample set of test vectors for Algo-Level1.

```
count = 0
seed = 3A8F...
pk = ...
sk = ...
ct = ...
ss = ...
```